

## Problem: 303. Range Sum Query - Immutable - Easy

```
class NumArray:
    def __init__(self, nums):
        self.prefix = [0]
        for value in nums:
            self.prefix.append(self.prefix[-1] + value)
    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

## Problem: 724. Find Pivot Index - Easy

```
class Solution:
    def pivotIndex(self, nums):
        total = sum(nums)
        left_sum = 0
        for index, value in enumerate(nums):
            right_sum = total - left_sum - value
            if left_sum == right_sum:
                return index
            left_sum += value
        return -1
```

## Problem: 1480. Running Sum of 1d Array - Easy

```
class Solution:
    def runningSum(self, nums):
        total = 0
        result = []
        for value in nums:
            total += value
            result.append(total)
        return result
```

## Problem: 1732. Find the Highest Altitude - Easy

```
class Solution:
    def largestAltitude(self, gain):
        altitude = 0
        best = 0
        for change in gain:
            altitude += change
```

```
        best = max(best, altitude)
    return best
```

## Problem: 1991. Find the Middle Index in Array - Easy

```
class Solution:
    def findMiddleIndex(self, nums):
        total = sum(nums)
        left_sum = 0
        for index, value in enumerate(nums):
            if left_sum == total - left_sum - value:
                return index
            left_sum += value
        return -1
```

## Problem: 2389. Longest Subsequence With Limited Sum - Easy

```
from bisect import bisect_right
class Solution:
    def answerQueries(self, nums, queries):
        nums.sort()
        prefix = [0]
        for value in nums:
            prefix.append(prefix[-1] + value)
        return [bisect_right(prefix, query) - 1 for query in queries]
```

## Problem: 560. Subarray Sum Equals K - Medium

```
class Solution:
    def subarraySum(self, nums, k):
        counts = {0: 1}
        prefix = 0
        answer = 0
        for value in nums:
            prefix += value
            answer += counts.get(prefix - k, 0)
            counts[prefix] = counts.get(prefix, 0) + 1
        return answer
```

## Problem: 523. Continuous Subarray Sum - Medium

```
class Solution:
    def checkSubarraySum(self, nums, k):
        first_index = {0: -1}
```

```
remainder = 0
for index, value in enumerate(nums):
    remainder = (remainder + value) % k
    if remainder in first_index:
        if index - first_index[remainder] >= 2:
            return True
    else:
        first_index[remainder] = index
return False
```

## Problem: 525. Contiguous Array - Medium

```
class Solution:
    def findMaxLength(self, nums):
        first_index = {0: -1}
        balance = 0
        best = 0
        for index, value in enumerate(nums):
            balance += 1 if value == 1 else -1
            if balance in first_index:
                best = max(best, index - first_index[balance])
            else:
                first_index[balance] = index
        return best
```

## Problem: 930. Binary Subarrays With Sum - Medium

```
class Solution:
    def numSubarraysWithSum(self, nums, goal):
        counts = {0: 1}
        prefix = 0
        answer = 0
        for value in nums:
            prefix += value
            answer += counts.get(prefix - goal, 0)
            counts[prefix] = counts.get(prefix, 0) + 1
        return answer
```

## Problem: 974. Subarray Sums Divisible by K - Medium

```
from collections import defaultdict
class Solution:
    def subarraysDivByK(self, nums, k):
        counts = defaultdict(int)
        counts[0] = 1
        remainder = 0
```

```
answer = 0
for value in nums:
    remainder = (remainder + value) % k
    answer += counts[remainder]
    counts[remainder] += 1
return answer
```

## Problem: 1248. Count Number of Nice Subarrays - Medium

```
class Solution:
    def numberOfSubarrays(self, nums, k):
        counts = {0: 1}
        odd_count = 0
        answer = 0
        for value in nums:
            odd_count += value % 2
            answer += counts.get(odd_count - k, 0)
            counts[odd_count] = counts.get(odd_count, 0) + 1
        return answer
```

## Problem: 1590. Make Sum Divisible by P - Medium

```
class Solution:
    def minSubarray(self, nums, p):
        total_remainder = sum(nums) % p
        if total_remainder == 0:
            return 0
        last_index = {0: -1}
        remainder = 0
        best = len(nums)
        for index, value in enumerate(nums):
            remainder = (remainder + value) % p
            needed = (remainder - total_remainder) % p
            if needed in last_index:
                best = min(best, index - last_index[needed])
            last_index[remainder] = index
        return -1 if best == len(nums) else best
```

## Problem: 1124. Longest Well-Performing Interval - Medium

```
class Solution:
    def longestWPI(self, hours):
        first_index = {0: -1}
        score = 0
        best = 0
        for index, hour in enumerate(hours):
```

```
score += 1 if hour > 8 else -1
if score > 0:
    best = index + 1
else:
    if score not in first_index:
        first_index[score] = index
    if score - 1 in first_index:
        best = max(best, index - first_index[score - 1])
return best
```

## Problem: 1442. Count Triplets That Can Form Two Arrays of Equal XOR - Medium

```
class Solution:
    def countTriplets(self, arr):
        prefix = [0]
        for value in arr:
            prefix.append(prefix[-1] ^ value)
        answer = 0
        for i in range(len(arr)):
            for k in range(i + 1, len(arr)):
                if prefix[i] == prefix[k + 1]:
                    answer += k - i
        return answer
```

## Problem: 1524. Number of Sub-arrays With Odd Sum - Medium

```
class Solution:
    def numOfSubarrays(self, arr):
        even = 1
        odd = 0
        parity = 0
        answer = 0
        for value in arr:
            parity ^= value & 1
            if parity:
                answer += even
                odd += 1
            else:
                answer += odd
                even += 1
        return answer % (10**9 + 7)
```

## Problem: 304. Range Sum Query 2D - Immutable - Medium

```

class NumMatrix:
    def __init__(self, matrix):
        rows = len(matrix)
        columns = len(matrix[0]) if rows else 0
        self.prefix = [[0] * (columns + 1) for _ in range(rows + 1)]
        for row in range(rows):
            for column in range(columns):
                self.prefix[row + 1][column + 1] = (
                    matrix[row][column]
                    + self.prefix[row][column + 1]
                    + self.prefix[row + 1][column]
                    - self.prefix[row][column]
                )
    def sumRegion(self, row1, col1, row2, col2):
        return (
            self.prefix[row2 + 1][col2 + 1]
            - self.prefix[row1][col2 + 1]
            - self.prefix[row2 + 1][col1]
            + self.prefix[row1][col1]
        )

```

## Problem: 1314. Matrix Block Sum - Medium

```

class Solution:
    def matrixBlockSum(self, mat, k):
        rows, columns = len(mat), len(mat[0])
        prefix = [[0] * (columns + 1) for _ in range(rows + 1)]
        for row in range(rows):
            for column in range(columns):
                prefix[row + 1][column + 1] = (
                    mat[row][column]
                    + prefix[row][column + 1]
                    + prefix[row + 1][column]
                    - prefix[row][column]
                )
        result = [[0] * columns for _ in range(rows)]
        for row in range(rows):
            for column in range(columns):
                top = max(0, row - k)
                bottom = min(rows - 1, row + k)
                left = max(0, column - k)
                right = min(columns - 1, column + k)
                result[row][column] = (
                    prefix[bottom + 1][right + 1]
                    - prefix[top][right + 1]
                    - prefix[bottom + 1][left]
                    + prefix[top][left]
                )
        return result

```

## Problem: 1074. Number of Submatrices That Sum to Target - Hard

```
from collections import defaultdict
class Solution:
    def numSubmatrixSumTarget(self, matrix, target):
        rows, columns = len(matrix), len(matrix[0])
        answer = 0
        for left in range(columns):
            row_sums = [0] * rows
            for right in range(left, columns):
                for row in range(rows):
                    row_sums[row] += matrix[row][right]
                counts = defaultdict(int)
                counts[0] = 1
                prefix = 0
                for value in row_sums:
                    prefix += value
                    answer += counts[prefix - target]
                    counts[prefix] += 1
        return answer
```

## Problem: 1109. Corporate Flight Bookings - Medium

```
class Solution:
    def corpFlightBookings(self, bookings, n):
        difference = [0] * (n + 1)
        for first, last, seats in bookings:
            difference[first - 1] += seats
            difference[last] -= seats
        answer = []
        current = 0
        for index in range(n):
            current += difference[index]
            answer.append(current)
        return answer
```

## Problem: 1854. Maximum Population Year - Easy

```
class Solution:
    def maximumPopulation(self, logs):
        difference = [0] * 102
        for birth, death in logs:
            difference[birth - 1950] += 1
            difference[death - 1950] -= 1
        current = 0
        best_population = 0
        best_year = 1950
```

```

for offset, change in enumerate(difference[::-1]):
    current += change
    if current > best_population:
        best_population = current
        best_year = 1950 + offset
return best_year

```

## Problem: 1893. Check if All the Integers in a Range Are Covered - Easy

```

class Solution:
    def isCovered(self, ranges, left, right):
        difference = [0] * 52
        for start, end in ranges:
            difference[start] += 1
            difference[end + 1] -= 1
        coverage = 0
        for value in range(1, right + 1):
            coverage += difference[value]
            if value >= left and coverage == 0:
                return False
        return True

```

## Problem: 437. Path Sum III - Medium

```

from collections import defaultdict
class Solution:
    def pathSum(self, root, targetSum):
        prefix_counts = defaultdict(int)
        prefix_counts[0] = 1
        def dfs(node, prefix):
            if not node:
                return 0
            prefix += node.val
            answer = prefix_counts[prefix - targetSum]
            prefix_counts[prefix] += 1
            answer += dfs(node.left, prefix)
            answer += dfs(node.right, prefix)
            prefix_counts[prefix] -= 1
            return answer
        return dfs(root, 0)

```

## Problem: 1109. Corporate Flight Bookings - Medium

```

class Solution:
    def corpFlightBookings(self, bookings, n):
        difference = [0] * (n + 1)

```

```

for first, last, seats in bookings:
    difference[first - 1] += seats
    difference[last] -= seats
answer = []
current = 0
for index in range(n):
    current += difference[index]
    answer.append(current)
return answer

```

## Problem: 1589. Maximum Sum Obtained of Any Permutation - Medium

```

class Solution:
    def maxSumRangeQuery(self, nums, requests):
        difference = [0] * (len(nums) + 1)
        for start, end in requests:
            difference[start] += 1
            difference[end + 1] -= 1
        frequency = []
        current = 0
        for index in range(len(nums)):
            current += difference[index]
            frequency.append(current)
        nums.sort()
        frequency.sort()
        modulo = 10**9 + 7
        return sum(value * count for value, count in zip(nums, frequency)) %
modulo

```

## Problem: 1594. Maximum Non Negative Product in a Matrix - Medium

```

class Solution:
    def maxProductPath(self, grid):
        rows, columns = len(grid), len(grid[0])
        minimum = [[0] * columns for _ in range(rows)]
        maximum = [[0] * columns for _ in range(rows)]
        minimum[0][0] = maximum[0][0] = grid[0][0]
        for row in range(rows):
            for column in range(columns):
                if row == 0 and column == 0:
                    continue
                candidates = []
                if row > 0:
                    candidates.extend((minimum[row - 1][column] * grid[row]
[column],
                                     maximum[row - 1][column] * grid[row]
[column]))
                if column > 0:
                    candidates.extend((minimum[row][column - 1] * grid[row]

```

```

[column],
                                maximum[row][column - 1] * grid[row]
[column]))
    minimum[row][column] = min(candidates)
    maximum[row][column] = max(candidates)
answer = maximum[-1][-1]
return answer % (10**9 + 7) if answer >= 0 else -1

```

## Problem: 862. Shortest Subarray with Sum at Least K - Hard

```

from collections import deque
class Solution:
    def shortestSubarray(self, nums, k):
        prefix = [0]
        for value in nums:
            prefix.append(prefix[-1] + value)
        candidates = deque()
        best = len(nums) + 1
        for right, total in enumerate(prefix):
            while candidates and total - prefix[candidates[0]] >= k:
                best = min(best, right - candidates.popleft())
            while candidates and prefix[candidates[-1]] >= total:
                candidates.pop()
            candidates.append(right)
        return -1 if best == len(nums) + 1 else best

```

## Problem: 77. Combinations - Medium

```

class Solution:
    def combine(self, n, k):
        result = []
        def backtrack(start, path):
            if len(path) == k:
                result.append(path[:])
                return
            for value in range(start, n + 1):
                path.append(value)
                backtrack(value + 1, path)
                path.pop()
        backtrack(1, [])
        return result

```

## Problem: 78. Subsets - Medium

```

class Solution:
    def subsets(self, nums):

```

```
result = []
def backtrack(index, path):
    if index == len(nums):
        result.append(path[:])
        return
    backtrack(index + 1, path)
    path.append(nums[index])
    backtrack(index + 1, path)
    path.pop()
backtrack(0, [])
return result
```

## Problem: 90. Subsets II - Medium

```
class Solution:
    def subsetsWithDup(self, nums):
        nums.sort()
        result = []
        def backtrack(start, path):
            result.append(path[:])
            for index in range(start, len(nums)):
                if index > start and nums[index] == nums[index - 1]:
                    continue
                path.append(nums[index])
                backtrack(index + 1, path)
                path.pop()
        backtrack(0, [])
        return result
```

## Problem: 39. Combination Sum - Medium

```

class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, remaining, path):
            if remaining == 0:
                result.append(path[:])
                return
            for index in range(start, len(candidates)):
                value = candidates[index]
                if value > remaining:
                    break
                path.append(value)
                backtrack(index, remaining - value, path)
                path.pop()
        backtrack(0, target, [])
        return result

```

## Problem: 40. Combination Sum II - Medium

```

class Solution:
    def combinationSum2(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, remaining, path):
            if remaining == 0:
                result.append(path[:])
                return
            for index in range(start, len(candidates)):
                if index > start and candidates[index] == candidates[index - 1]:
                    continue
                if candidates[index] > remaining:
                    break
                path.append(candidates[index])
                backtrack(index + 1, remaining - candidates[index], path)
                path.pop()
        backtrack(0, target, [])
        return result

```

## Problem: 216. Combination Sum III - Medium

```

class Solution:
    def combinationSum3(self, k, n):
        result = []
        def backtrack(start, remaining, path):
            if len(path) == k:
                if remaining == 0:

```

```

        result.append(path[:])
    return
    for value in range(start, 10):
        if value > remaining:
            break
        path.append(value)
        backtrack(value + 1, remaining - value, path)
        path.pop()
    backtrack(1, n, [])
    return result

```

## Problem: 46. Permutations - Medium

```

class Solution:
    def permute(self, nums):
        result = []
        def backtrack(path, used):
            if len(path) == len(nums):
                result.append(path[:])
                return
            for index, value in enumerate(nums):
                if used[index]:
                    continue
                used[index] = True
                path.append(value)
                backtrack(path, used)
                path.pop()
                used[index] = False
            backtrack([], [False] * len(nums))
        return result

```

## Problem: 47. Permutations II - Medium

```

class Solution:
    def permuteUnique(self, nums):
        nums.sort()
        result = []
        def backtrack(path, used):
            if len(path) == len(nums):
                result.append(path[:])
                return
            for index, value in enumerate(nums):
                if used[index] or (index > 0 and nums[index] == nums[index - 1]
and not used[index - 1]):
                    continue
                used[index] = True
                path.append(value)
                backtrack(path, used)
                path.pop()

```

```

        used[index] = False
    backtrack([], [False] * len(nums))
    return result

```

## Problem: 60. Permutation Sequence - Hard

```

import math
class Solution:
    def getPermutation(self, n, k):
        numbers = list(range(1, n + 1))
        result = []
        k -= 1
        for remaining in range(n, 0, -1):
            block = math.factorial(remaining - 1)
            index, k = divmod(k, block)
            result.append(str(numbers.pop(index)))
        return ''.join(result)

```

## Problem: 17. Letter Combinations of a Phone Number - Medium

```

class Solution:
    def letterCombinations(self, digits):
        if not digits:
            return []
        mapping = {
            '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
            '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
        }
        result = []
        def backtrack(index, path):
            if index == len(digits):
                result.append(''.join(path))
                return
            for character in mapping[digits[index]]:
                path.append(character)
                backtrack(index + 1, path)
                path.pop()
        backtrack(0, [])
        return result

```

## Problem: 22. Generate Parentheses - Medium

```

class Solution:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, opened, closed):

```

```

        if len(path) == 2 * n:
            result.append(''.join(path))
            return
        if opened < n:
            path.append('(')
            backtrack(path, opened + 1, closed)
            path.pop()
        if closed < opened:
            path.append(')')
            backtrack(path, opened, closed + 1)
            path.pop()
    backtrack([], 0, 0)
    return result

```

## Problem: 131. Palindrome Partitioning - Medium

```

class Solution:
    def partition(self, s):
        result = []
        def backtrack(start, path):
            if start == len(s):
                result.append(path[:])
                return
            for end in range(start, len(s)):
                piece = s[start:end + 1]
                if piece != piece[::-1]:
                    continue
                path.append(piece)
                backtrack(end + 1, path)
                path.pop()
        backtrack(0, [])
        return result

```

## Problem: 93. Restore IP Addresses - Medium

```

class Solution:
    def restoreIpAddresses(self, s):
        result = []
        def backtrack(index, path):
            remaining_parts = 4 - len(path)
            remaining_characters = len(s) - index
            if remaining_characters < remaining_parts or remaining_characters > 3 * remaining_parts:
                return
            if len(path) == 4:
                if index == len(s):
                    result.append('.'.join(path))
                return
            for end in range(index, min(index + 3, len(s))):

```

```

        piece = s[index:end + 1]
        if len(piece) > 1 and piece[0] == '0':
            break
        if int(piece) > 255:
            continue
        path.append(piece)
        backtrack(end + 1, path)
        path.pop()
    backtrack(0, [])
    return result

```

## Problem: 140. Word Break II - Hard

```

from functools import lru_cache
class Solution:
    def wordBreak(self, s, wordDict):
        words = set(wordDict)
        @lru_cache(None)
        def build(start):
            if start == len(s):
                return ['']
            result = []
            for end in range(start + 1, len(s) + 1):
                word = s[start:end]
                if word not in words:
                    continue
                for suffix in build(end):
                    result.append(word if not suffix else word + ' ' + suffix)
            return result
        return build(0)

```

## Problem: 37. Sudoku Solver - Hard

```

class Solution:
    def solveSudoku(self, board):
        def is_valid(row, column, value):
            for index in range(9):
                if board[row][index] == value or board[index][column] == value:
                    return False
            box_row = 3 * (row // 3) + index // 3
            box_column = 3 * (column // 3) + index % 3
            if board[box_row][box_column] == value:
                return False
            return True
        def solve():
            for row in range(9):
                for column in range(9):
                    if board[row][column] != '.':
                        continue

```

```

        for value in '123456789':
            if is_valid(row, column, value):
                board[row][column] = value
                if solve():
                    return True
                board[row][column] = '.'
            return False
        return True
    solve()

```

## Problem: 51. N-Queens - Hard

```

class Solution:
    def solveNQueens(self, n):
        result = []
        columns = set()
        diagonals = set()
        anti_diagonals = set()
        board = [['.'] * n for _ in range(n)]
        def backtrack(row):
            if row == n:
                result.append([''.join(line) for line in board])
                return
            for column in range(n):
                if column in columns or row - column in diagonals or row + column
in anti_diagonals:
                    continue
                columns.add(column)
                diagonals.add(row - column)
                anti_diagonals.add(row + column)
                board[row][column] = 'Q'
                backtrack(row + 1)
                board[row][column] = '.'
                columns.remove(column)
                diagonals.remove(row - column)
                anti_diagonals.remove(row + column)
        backtrack(0)
        return result

```

## Problem: 52. N-Queens II - Hard

```

class Solution:
    def totalNQueens(self, n):
        columns = set()
        diagonals = set()
        anti_diagonals = set()
        def count(row):
            if row == n:
                return 1

```

```

        answer = 0
        for column in range(n):
            if column in columns or row - column in diagonals or row + column
in anti_diagonals:
                continue
            columns.add(column)
            diagonals.add(row - column)
            anti_diagonals.add(row + column)
            answer += count(row + 1)
            columns.remove(column)
            diagonals.remove(row - column)
            anti_diagonals.remove(row + column)
        return answer
    return count(0)

```

## Problem: 79. Word Search - Medium

```

class Solution:
    def exist(self, board, word):
        rows, columns = len(board), len(board[0])
        def search(row, column, index):
            if index == len(word):
                return True
            if not (0 <= row < rows and 0 <= column < columns):
                return False
            if board[row][column] != word[index]:
                return False
            saved = board[row][column]
            board[row][column] = '#'
            found = any(search(row + dr, column + dc, index + 1)
                        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)))
            board[row][column] = saved
            return found
        return any(search(row, column, 0)
                   for row in range(rows) for column in range(columns))

```

## Problem: 980. Unique Paths III - Hard

```

class Solution:
    def uniquePathsIII(self, grid):
        rows, columns = len(grid), len(grid[0])
        empty = 0
        start = None
        for row in range(rows):
            for column in range(columns):
                if grid[row][column] != -1:
                    empty += 1
                if grid[row][column] == 1:
                    start = (row, column)

```

```

def backtrack(row, column, remaining):
    if not (0 <= row < rows and 0 <= column < columns) or grid[row]
[column] == -1:
        return 0
    if grid[row][column] == 2:
        return int(remaining == 1)
    value = grid[row][column]
    grid[row][column] = -1
    answer = sum(backtrack(row + dr, column + dc, remaining - 1)
                 for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)))
    grid[row][column] = value
    return answer
return backtrack(start[0], start[1], empty)

```

## Problem: 1255. Maximum Score Words Formed by Letters - Hard

```

from collections import Counter
class Solution:
    def maxScoreWords(self, words, letters, score):
        available = Counter(letters)
        def backtrack(index):
            if index == len(words):
                return 0
            best = backtrack(index + 1)
            needed = Counter(words[index])
            if all(available[character] >= count for character, count in
needed.items()):
                for character, count in needed.items():
                    available[character] -= count
                value = sum(score[ord(character) - ord('a')] for character in
words[index])
                best = max(best, value + backtrack(index + 1))
                for character, count in needed.items():
                    available[character] += count
            return best
        return backtrack(0)

```

## Problem: 212. Word Search II - Hard

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None
class Solution:
    def findWords(self, board, words):
        root = TrieNode()
        for word in words:
            node = root
            for character in word:

```

```

        node = node.children.setdefault(character, TrieNode())
        node.word = word
    rows, columns = len(board), len(board[0])
    result = []
    def search(row, column, node):
        if not (0 <= row < rows and 0 <= column < columns):
            return
        character = board[row][column]
        if character == '#' or character not in node.children:
            return
        next_node = node.children[character]
        if next_node.word:
            result.append(next_node.word)
            next_node.word = None
        board[row][column] = '#'
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            search(row + dr, column + dc, next_node)
        board[row][column] = character
    for row in range(rows):
        for column in range(columns):
            search(row, column, root)
    return result

```

## Problem: 282. Expression Add Operators - Hard

```

class Solution:
    def addOperators(self, num, target):
        result = []
        def backtrack(index, expression, value, previous):
            if index == len(num):
                if value == target:
                    result.append(expression)
                return
            for end in range(index, len(num)):
                if end > index and num[index] == '0':
                    break
                part = num[index:end + 1]
                current = int(part)
                if index == 0:
                    backtrack(end + 1, part, current, current)
                else:
                    backtrack(end + 1, expression + '+' + part, value + current,
current)
                    backtrack(end + 1, expression + '-' + part, value - current, -
current)
                    multiplied = previous * current
                    backtrack(end + 1, expression + '*' + part,
value - previous + multiplied, multiplied)
            backtrack(0, '', 0, 0)
        return result

```

## Problem: 31. Next Permutation - Medium

```
class Solution:
    def nextPermutation(self, nums):
        pivot = len(nums) - 2
        while pivot >= 0 and nums[pivot] >= nums[pivot + 1]:
            pivot -= 1
        if pivot >= 0:
            successor = len(nums) - 1
            while nums[successor] <= nums[pivot]:
                successor -= 1
            nums[pivot], nums[successor] = nums[successor], nums[pivot]
        left, right = pivot + 1, len(nums) - 1
        while left < right:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

## Problem: 667. Beautiful Arrangement II - Medium

```
class Solution:
    def constructArray(self, n, k):
        result = []
        left, right = 1, n
        while left <= right:
            if k > 1:
                if k % 2:
                    result.append(left)
                    left += 1
                else:
                    result.append(right)
                    right -= 1
                k -= 1
            else:
                result.append(left)
                left += 1
        return result
```

## Problem: 50. Pow(x, n) - Medium

```
class Solution:
    def myPow(self, x, n):
        if n == 0:
            return 1.0
        if n < 0:
            return 1 / self.myPow(x, -n)
        half = self.myPow(x, n // 2)
```

```
if n % 2 == 0:
    return half * half
return half * half * x
```

## Problem: 70. Climbing Stairs - Easy

```
class Solution:
    def climbStairs(self, n):
        if n <= 2:
            return n
        return self.climbStairs(n - 1) + self.climbStairs(n - 2)
```

## Problem: 509. Fibonacci Number - Easy

```
class Solution:
    def fib(self, n):
        if n <= 1:
            return n
        return self.fib(n - 1) + self.fib(n - 2)
```

## Problem: 779. K-th Symbol in Grammar - Medium

```
class Solution:
    def kthGrammar(self, n, k):
        if n == 1:
            return 0
        parent = self.kthGrammar(n - 1, (k + 1) // 2)
        if k % 2 == 1:
            return parent
        return 1 - parent
```

## Problem: 231. Power of Two - Easy

```
class Solution:
    def isPowerOfTwo(self, n):
        if n == 1:
            return True
        if n <= 0 or n % 2:
            return False
        return self.isPowerOfTwo(n // 2)
```

## Problem: 94. Binary Tree Inorder Traversal - Easy

```
class Solution:
    def inorderTraversal(self, root):
        if not root:
            return []
        return self.inorderTraversal(root.left) + [root.val] +
self.inorderTraversal(root.right)
```

## Problem: 100. Same Tree - Easy

```
class Solution:
    def isSameTree(self, p, q):
        if not p and not q:
            return True
        if not p or not q or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right,
q.right)
```

## Problem: 104. Maximum Depth of Binary Tree - Easy

```
class Solution:
    def maxDepth(self, root):
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

## Problem: 226. Invert Binary Tree - Easy

```
class Solution:
    def invertTree(self, root):
        if not root:
            return None
        root.left, root.right = self.invertTree(root.right),
self.invertTree(root.left)
        return root
```

## Problem: 236. Lowest Common Ancestor of a Binary Tree - Medium

```
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if not root or root == p or root == q:
            return root
```

```
left = self.lowestCommonAncestor(root.left, p, q)
right = self.lowestCommonAncestor(root.right, p, q)
if left and right:
    return root
return left if left else right
```

## Problem: 543. Diameter of Binary Tree - Easy

```
class Solution:
    def diameterOfBinaryTree(self, root):
        best = 0
        def height(node):
            nonlocal best
            if not node:
                return 0
            left = height(node.left)
            right = height(node.right)
            best = max(best, left + right)
            return 1 + max(left, right)
        height(root)
        return best
```

## Problem: 124. Binary Tree Maximum Path Sum - Hard

```
class Solution:
    def maxPathSum(self, root):
        best = float('-inf')
        def gain(node):
            nonlocal best
            if not node:
                return 0
            left = max(0, gain(node.left))
            right = max(0, gain(node.right))
            best = max(best, node.val + left + right)
            return node.val + max(left, right)
        gain(root)
        return best
```

## Problem: 17. Letter Combinations of a Phone Number - Medium

```
class Solution:
    def letterCombinations(self, digits):
        if not digits:
            return []
        letters = {
            '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
```

```

        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }
    result = []
    def backtrack(index, path):
        if index == len(digits):
            result.append(''.join(path))
            return
        for character in letters[digits[index]]:
            path.append(character)
            backtrack(index + 1, path)
            path.pop()
    backtrack(0, [])
    return result

```

## Problem: 22. Generate Parentheses - Medium

```

class Solution:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, opened, closed):
            if len(path) == 2 * n:
                result.append(''.join(path))
                return
            if opened < n:
                path.append('(')
                backtrack(path, opened + 1, closed)
                path.pop()
            if closed < opened:
                path.append(')')
                backtrack(path, opened, closed + 1)
                path.pop()
        backtrack([], 0, 0)
        return result

```

## Problem: 39. Combination Sum - Medium

```

class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, remaining, path):
            if remaining == 0:
                result.append(path[:])
                return
            for index in range(start, len(candidates)):
                value = candidates[index]
                if value > remaining:
                    break
                path.append(value)

```

```
        backtrack(index, remaining - value, path)
        path.pop()
    backtrack(0, target, [])
    return result
```

## Problem: 46. Permutations - Medium

```
class Solution:
    def permute(self, nums):
        result = []
        def backtrack(path, used):
            if len(path) == len(nums):
                result.append(path[:])
                return
            for index, value in enumerate(nums):
                if used[index]:
                    continue
                used[index] = True
                path.append(value)
                backtrack(path, used)
                path.pop()
                used[index] = False
            backtrack([], [False] * len(nums))
        return result
```

## Problem: 47. Permutations II - Medium

```
class Solution:
    def permuteUnique(self, nums):
        nums.sort()
        result = []
        def backtrack(path, used):
            if len(path) == len(nums):
                result.append(path[:])
                return
            for index, value in enumerate(nums):
                if used[index] or (index > 0 and nums[index] == nums[index - 1]
and not used[index - 1]):
                    continue
                used[index] = True
                path.append(value)
                backtrack(path, used)
                path.pop()
                used[index] = False
            backtrack([], [False] * len(nums))
        return result
```

## Problem: 78. Subsets - Medium

```
class Solution:
    def subsets(self, nums):
        result = []
        def backtrack(index, path):
            if index == len(nums):
                result.append(path[:])
                return
            backtrack(index + 1, path)
            path.append(nums[index])
            backtrack(index + 1, path)
            path.pop()
        backtrack(0, [])
        return result
```

## Problem: 90. Subsets II - Medium

```
class Solution:
    def subsetsWithDup(self, nums):
        nums.sort()
        result = []
        def backtrack(start, path):
            result.append(path[:])
            for index in range(start, len(nums)):
                if index > start and nums[index] == nums[index - 1]:
                    continue
                path.append(nums[index])
                backtrack(index + 1, path)
                path.pop()
        backtrack(0, [])
        return result
```

## Problem: 79. Word Search - Medium

```
class Solution:
    def exist(self, board, word):
        rows, columns = len(board), len(board[0])
        def search(row, column, index):
            if index == len(word):
                return True
            if not (0 <= row < rows and 0 <= column < columns):
                return False
            if board[row][column] != word[index]:
                return False
            saved = board[row][column]
            board[row][column] = '#'
            if search(row, column, index + 1):
                return True
            board[row][column] = saved
            return False
        for row in range(rows):
            for column in range(columns):
                if search(row, column, 0):
                    return True
        return False
```

```

        found = any(search(row + dr, column + dc, index + 1)
                     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)))
        board[row][column] = saved
        return found
    return any(search(row, column, 0)
               for row in range(rows) for column in range(columns))

```

## Problem: 53. Maximum Subarray - Medium

```

class Solution:
    def maxSubArray(self, nums):
        def divide(left, right):
            if left == right:
                return nums[left], nums[left], nums[left], nums[left]
            middle = (left + right) // 2
            left_sum, left_prefix, left_suffix, left_best = divide(left, middle)
            right_sum, right_prefix, right_suffix, right_best = divide(middle + 1,
right)

            total = left_sum + right_sum
            prefix = max(left_prefix, left_sum + right_prefix)
            suffix = max(right_suffix, right_sum + left_suffix)
            best = max(left_best, right_best, left_suffix + right_prefix)
            return total, prefix, suffix, best
        return divide(0, len(nums) - 1)[3]

```

## Problem: 108. Convert Sorted Array to Binary Search Tree - Easy

```

class Solution:
    def sortedArrayToBST(self, nums):
        def build(left, right):
            if left > right:
                return None
            middle = (left + right) // 2
            node = TreeNode(nums[middle])
            node.left = build(left, middle - 1)
            node.right = build(middle + 1, right)
            return node
        return build(0, len(nums) - 1)

```

## Problem: 148. Sort List - Medium

```

class Solution:
    def sortList(self, head):
        if not head or not head.next:
            return head
        slow, fast = head, head.next

```

```

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        second = slow.next
        slow.next = None
        left = self.sortList(head)
        right = self.sortList(second)
        return self.merge(left, right)
    def merge(self, left, right):
        dummy = ListNode(0)
        tail = dummy
        while left and right:
            if left.val <= right.val:
                tail.next, left = left, left.next
            else:
                tail.next, right = right, right.next
            tail = tail.next
        tail.next = left or right
        return dummy.next

```

## Problem: 912. Sort an Array - Medium

```

class Solution:
    def sortArray(self, nums):
        if len(nums) <= 1:
            return nums
        middle = len(nums) // 2
        left = self.sortArray(nums[:middle])
        right = self.sortArray(nums[middle:])
        result = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
        result.extend(left[i:])
        result.extend(right[j:])
        return result

```

## Problem: 215. Kth Largest Element in an Array - Medium

```

class Solution:
    def findKthLargest(self, nums, k):
        target = len(nums) - k
        def select(left, right):
            pivot = nums[right]

```

```

        store = left
        for index in range(left, right):
            if nums[index] <= pivot:
                nums[store], nums[index] = nums[index], nums[store]
                store += 1
        nums[store], nums[right] = nums[right], nums[store]
        if store == target:
            return nums[store]
        if store < target:
            return select(store + 1, right)
        return select(left, store - 1)
    return select(0, len(nums) - 1)

```

## Problem: 198. House Robber - Medium

```

from functools import lru_cache
class Solution:
    def rob(self, nums):
        @lru_cache(None)
        def best(index):
            if index >= len(nums):
                return 0
            return max(best(index + 1), nums[index] + best(index + 2))
        return best(0)

```

## Problem: 300. Longest Increasing Subsequence - Medium

```

from functools import lru_cache
class Solution:
    def lengthOfLIS(self, nums):
        @lru_cache(None)
        def length(index):
            best = 1
            for next_index in range(index + 1, len(nums)):
                if nums[next_index] > nums[index]:
                    best = max(best, 1 + length(next_index))
            return best
        return max((length(index) for index in range(len(nums))), default=0)

```

## Problem: 329. Longest Increasing Path in a Matrix - Hard

```

from functools import lru_cache
class Solution:
    def longestIncreasingPath(self, matrix):
        rows, columns = len(matrix), len(matrix[0])
        @lru_cache(None)

```

```
def path(row, column):
    best = 1
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        next_row, next_column = row + dr, column + dc
        if (0 <= next_row < rows and 0 <= next_column < columns
            and matrix[next_row][next_column] > matrix[row][column]):
            best = max(best, 1 + path(next_row, next_column))
    return best
return max(path(row, column) for row in range(rows) for column in
            range(columns))
```

## Problem: 115. Distinct Subsequences - Hard

```
from functools import lru_cache
class Solution:
    def numDistinct(self, s, t):
        @lru_cache(None)
        def count(i, j):
            if j == len(t):
                return 1
            if i == len(s):
                return 0
            answer = count(i + 1, j)
            if s[i] == t[j]:
                answer += count(i + 1, j + 1)
            return answer
        return count(0, 0)
```

## Problem: 148. Sort List - Medium

```
class Solution:
    def sortList(self, head):
        if not head or not head.next:
            return head
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        second = slow.next
        slow.next = None
        left = self.sortList(head)
        right = self.sortList(second)
        return self.merge(left, right)
    def merge(self, left, right):
        dummy = ListNode(0)
        tail = dummy
        while left and right:
            if left.val <= right.val:
                tail.next = left
```

```

        left = left.next
    else:
        tail.next = right
        right = right.next
        tail = tail.next
    tail.next = left if left else right
    return dummy.next

```

## Problem: 912. Sort an Array - Medium

```

class Solution:
    def sortArray(self, nums):
        if len(nums) <= 1:
            return nums
        middle = len(nums) // 2
        left = self.sortArray(nums[:middle])
        right = self.sortArray(nums[middle:])
        result = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
        result.extend(left[i:])
        result.extend(right[j:])
        return result

```

## Problem: 315. Count of Smaller Numbers After Self - Hard

```

class Solution:
    def countSmaller(self, nums):
        indexes = list(range(len(nums)))
        counts = [0] * len(nums)
        def merge_sort(left, right):
            if left >= right:
                return
            middle = (left + right) // 2
            merge_sort(left, middle)
            merge_sort(middle + 1, right)
            merged = []
            i, j = left, middle + 1
            while i <= middle and j <= right:
                if nums[indexes[i]] <= nums[indexes[j]]:
                    merged.append(indexes[i])
                    i += 1
                else:

```

```

        counts[indexes[j]] += middle - i + 1
        merged.append(indexes[j])
        j += 1
    merged.extend(indexes[i:middle + 1])
    merged.extend(indexes[j:right + 1])
    indexes[left:right + 1] = merged
    merge_sort(0, len(nums) - 1)
    return counts

```

## Problem: 493. Reverse Pairs - Hard

```

class Solution:
    def reversePairs(self, nums):
    def divide(left, right):
        if left >= right:
            return 0
        middle = (left + right) // 2
        answer = divide(left, middle) + divide(middle + 1, right)
        j = middle + 1
        for i in range(left, middle + 1):
            while j <= right and nums[i] > 2 * nums[j]:
                j += 1
            answer += j - (middle + 1)
        nums[left:right + 1] = sorted(nums[left:right + 1])
        return answer
    return divide(0, len(nums) - 1)

```

## Problem: 327. Count of Range Sum - Hard

```

class Solution:
    def countRangeSum(self, nums, lower, upper):
        prefix = [0]
        for value in nums:
            prefix.append(prefix[-1] + value)
        def divide(values):
            if len(values) <= 1:
                return 0, values
            middle = len(values) // 2
            left_count, left = divide(values[:middle])
            right_count, right = divide(values[middle:])
            answer = left_count + right_count
            low = high = 0
            for left_value in left:
                while low < len(right) and right[low] - left_value < lower:
                    low += 1
                while high < len(right) and right[high] - left_value <= upper:
                    high += 1
                answer += high - low
            merged = []

```

```

        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        merged.extend(left[i:])
        merged.extend(right[j:])
        return answer, merged
    return divide(prefix)[0]

```

## Problem: 53. Maximum Subarray - Medium

```

class Solution:
    def maxSubArray(self, nums):
        def divide(left, right):
            if left == right:
                value = nums[left]
                return value, value, value, value
            middle = (left + right) // 2
            left_total, left_prefix, left_suffix, left_best = divide(left, middle)
            right_total, right_prefix, right_suffix, right_best = divide(middle +
1, right)
            total = left_total + right_total
            prefix = max(left_prefix, left_total + right_prefix)
            suffix = max(right_suffix, right_total + left_suffix)
            best = max(left_best, right_best, left_suffix + right_prefix)
            return total, prefix, suffix, best
        return divide(0, len(nums) - 1)[3]

```

## Problem: 215. Kth Largest Element in an Array - Medium

```

class Solution:
    def findKthLargest(self, nums, k):
        target = len(nums) - k
        def select(left, right):
            pivot = nums[right]
            store = left
            for index in range(left, right):
                if nums[index] <= pivot:
                    nums[store], nums[index] = nums[index], nums[store]
                    store += 1
            nums[store], nums[right] = nums[right], nums[store]
            if store == target:
                return nums[store]
            if store < target:
                return select(store + 1, right)

```

```
        return select(left, store - 1)
    return select(0, len(nums) - 1)
```

## Problem: 33. Search in Rotated Sorted Array - Medium

```
class Solution:
    def search(self, nums, target):
        def binary(left, right):
            if left > right:
                return -1
            middle = (left + right) // 2
            if nums[middle] == target:
                return middle
            if nums[left] <= nums[middle]:
                if nums[left] <= target < nums[middle]:
                    return binary(left, middle - 1)
                return binary(middle + 1, right)
            if nums[middle] < target <= nums[right]:
                return binary(middle + 1, right)
            return binary(left, middle - 1)
        return binary(0, len(nums) - 1)
```

## Problem: 4. Median of Two Sorted Arrays - Hard

```
class Solution:
    def findMedianSortedArrays(self, nums1, nums2):
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        left, right = 0, len(nums1)
        total = len(nums1) + len(nums2)
        while left <= right:
            cut1 = (left + right) // 2
            cut2 = (total + 1) // 2 - cut1
            left1 = nums1[cut1 - 1] if cut1 else float('-inf')
            right1 = nums1[cut1] if cut1 < len(nums1) else float('inf')
            left2 = nums2[cut2 - 1] if cut2 else float('-inf')
            right2 = nums2[cut2] if cut2 < len(nums2) else float('inf')
            if left1 <= right2 and left2 <= right1:
                if total % 2:
                    return float(max(left1, left2))
                return (max(left1, left2) + min(right1, right2)) / 2
            if left1 > right2:
                right = cut1 - 1
            else:
                left = cut1 + 1
```

## Problem: 69. Sqrt(x) - Easy

```
class Solution:
    def mySqrt(self, x):
        def search(left, right):
            if left > right:
                return right
            middle = (left + right) // 2
            if middle * middle <= x:
                return search(middle + 1, right)
            return search(left, middle - 1)
        return search(0, x)
```

## Problem: 105. Construct Binary Tree from Preorder and Inorder Traversal - Medium

```
class Solution:
    def buildTree(self, preorder, inorder):
        positions = {value: index for index, value in enumerate(inorder)}
        preorder_index = 0
        def build(left, right):
            nonlocal preorder_index
            if left > right:
                return None
            value = preorder[preorder_index]
            preorder_index += 1
            node = TreeNode(value)
            middle = positions[value]
            node.left = build(left, middle - 1)
            node.right = build(middle + 1, right)
            return node
        return build(0, len(inorder) - 1)
```

## Problem: 106. Construct Binary Tree from Inorder and Postorder Traversal - Medium

```
class Solution:
    def buildTree(self, inorder, postorder):
        positions = {value: index for index, value in enumerate(inorder)}
        postorder_index = len(postorder) - 1
        def build(left, right):
            nonlocal postorder_index
            if left > right:
                return None
            value = postorder[postorder_index]
            postorder_index -= 1
            node = TreeNode(value)
            middle = positions[value]
            node.right = build(middle + 1, right)
```

```

        node.left = build(left, middle - 1)
        return node
    return build(0, len(inorder) - 1)

```

## Problem: 108. Convert Sorted Array to Binary Search Tree - Easy

```

class Solution:
    def sortedArrayToBST(self, nums):
        def build(left, right):
            if left > right:
                return None
            middle = (left + right) // 2
            node = TreeNode(nums[middle])
            node.left = build(left, middle - 1)
            node.right = build(middle + 1, right)
            return node
        return build(0, len(nums) - 1)

```

## Problem: 654. Maximum Binary Tree - Medium

```

class Solution:
    def constructMaximumBinaryTree(self, nums):
        def build(left, right):
            if left >= right:
                return None
            maximum_index = max(range(left, right), key=nums.__getitem__)
            node = TreeNode(nums[maximum_index])
            node.left = build(left, maximum_index)
            node.right = build(maximum_index + 1, right)
            return node
        return build(0, len(nums))

```

## Problem: 23. Merge k Sorted Lists - Hard

```

class Solution:
    def mergeKLists(self, lists):
        if not lists:
            return None
        if len(lists) == 1:
            return lists[0]
        middle = len(lists) // 2
        left = self.mergeKLists(lists[:middle])
        right = self.mergeKLists(lists[middle:])
        return self.mergeTwo(left, right)
    def mergeTwo(self, left, right):
        dummy = ListNode(0)

```

```

tail = dummy
while left and right:
    if left.val <= right.val:
        tail.next, left = left, left.next
    else:
        tail.next, right = right, right.next
    tail = tail.next
tail.next = left or right
return dummy.next

```

## Problem: 241. Different Ways to Add Parentheses - Medium

```

from functools import lru_cache
class Solution:
    def diffWaysToCompute(self, expression):
        @lru_cache(None)
        def compute(text):
            results = []
            for index, character in enumerate(text):
                if character not in '+-*':
                    continue
                left_values = compute(text[:index])
                right_values = compute(text[index + 1:])
                for left in left_values:
                    for right in right_values:
                        if character == '+':
                            results.append(left + right)
                        elif character == '-':
                            results.append(left - right)
                        else:
                            results.append(left * right)
            if not results:
                results.append(int(text))
            return results
        return compute(expression)

```

## Problem: 395. Longest Substring with At Least K Repeating Characters - Medium

```

from collections import Counter
class Solution:
    def longestSubstring(self, s, k):
        if len(s) < k:
            return 0
        counts = Counter(s)
        for index, character in enumerate(s):
            if counts[character] < k:
                return max(self.longestSubstring(s[:index], k),

```

```
self.longestSubstring(s[index + 1:], k))
return len(s)
```

## Problem: 427. Construct Quad Tree - Medium

```
class Solution:
    def construct(self, grid):
        def build(row, column, size):
            first = grid[row][column]
            uniform = all(grid[r][c] == first
                          for r in range(row, row + size)
                          for c in range(column, column + size))
            if uniform:
                return Node(first == 1, True)
            half = size // 2
            return Node(
                True,
                False,
                build(row, column, half),
                build(row, column + half, half),
                build(row + half, column, half),
                build(row + half, column + half, half)
            )
        return build(0, 0, len(grid))
```

## Problem: 932. Beautiful Array - Medium

```
from functools import lru_cache
class Solution:
    def beautifulArray(self, n):
        @lru_cache(None)
        def build(size):
            if size == 1:
                return (1,)
            odds = tuple(2 * value - 1 for value in build((size + 1) // 2))
            evens = tuple(2 * value for value in build(size // 2))
            return odds + evens
        return list(build(n))
```

## Problem: 240. Search a 2D Matrix II - Medium

```
class Solution:
    def searchMatrix(self, matrix, target):
        if not matrix or not matrix[0]:
            return False
        rows, columns = len(matrix), len(matrix[0])
```

```

def search(top, bottom, left, right):
    if top > bottom or left > right:
        return False
    if target < matrix[top][left] or target > matrix[bottom][right]:
        return False
    middle_row = (top + bottom) // 2
    middle_column = (left + right) // 2
    value = matrix[middle_row][middle_column]
    if value == target:
        return True
    if value > target:
        return (search(top, middle_row - 1, left, right)
                or search(middle_row, bottom, left, middle_column - 1))
    return (search(middle_row + 1, bottom, left, right)
            or search(top, middle_row, middle_column + 1, right))
return search(0, rows - 1, 0, columns - 1)

```

## Problem: 39. Combination Sum - Medium

```

class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, remaining, path):
            if remaining == 0:
                result.append(path[:])
                return
            for index in range(start, len(candidates)):
                value = candidates[index]
                if value > remaining:
                    break
                path.append(value)
                backtrack(index, remaining - value, path)
                path.pop()
            backtrack(0, target, [])
        return result

```

## Problem: 46. Permutations - Medium

```

class Solution:
    def permute(self, nums):
        result = []
        def backtrack(path, used):
            if len(path) == len(nums):
                result.append(path[:])
                return
            for index, value in enumerate(nums):
                if used[index]:
                    continue

```

```

        used[index] = True
        path.append(value)
        backtrack(path, used)
        path.pop()
        used[index] = False
    backtrack([], [False] * len(nums))
    return result

```

## Problem: 509. Fibonacci Number - Easy

```

class Solution:
    def fib(self, n):
        if n <= 1:
            return n
        return self.fib(n - 1) + self.fib(n - 2)

```

## Problem: 78. Subsets - Medium

```

class Solution:
    def subsets(self, nums):
        result = []
        def backtrack(index, path):
            if index == len(nums):
                result.append(path[:])
                return
            backtrack(index + 1, path)
            path.append(nums[index])
            backtrack(index + 1, path)
            path.pop()
        backtrack(0, [])
        return result

```

## Problem: 889. Construct Binary Tree from Preorder and Postorder Traversal - Medium

```

class Solution:
    def constructFromPrePost(self, preorder, postorder):
        if not preorder:
            return None
        root = TreeNode(preorder[0])
        if len(preorder) == 1:
            return root
        left_root = preorder[1]
        left_size = postorder.index(left_root) + 1
        root.left = self.constructFromPrePost(preorder[1:left_size + 1],
        postorder[:left_size])

```

```
        root.right = self.constructFromPrePost(preorder[left_size + 1:],
        postorder[left_size:-1])
        return root
```

## Problem: 102. Binary Tree Level Order Traversal - Medium

```
from collections import deque
class Solution:
    def levelOrder(self, root):
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level = []
            for _ in range(len(queue)):
                node = queue.popleft()
                level.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            result.append(level)
        return result
```

## Problem: 103. Binary Tree Zigzag Level Order Traversal - Medium

```
from collections import deque
class Solution:
    def zigzagLevelOrder(self, root):
        if not root:
            return []
        result = []
        queue = deque([root])
        reverse = False
        while queue:
            level = []
            for _ in range(len(queue)):
                node = queue.popleft()
                level.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            if reverse:
                level.reverse()
            result.append(level)
            reverse = not reverse
```

```
reverse = not reverse
return result
```

## Problem: 107. Binary Tree Level Order Traversal II - Medium

```
from collections import deque
class Solution:
    def levelOrderBottom(self, root):
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level = []
            for _ in range(len(queue)):
                node = queue.popleft()
                level.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            result.append(level)
        return result[::-1]
```

## Problem: 111. Minimum Depth of Binary Tree - Easy

```
from collections import deque
class Solution:
    def minDepth(self, root):
        if not root:
            return 0
        queue = deque([(root, 1)])
        while queue:
            node, depth = queue.popleft()
            if not node.left and not node.right:
                return depth
            if node.left:
                queue.append((node.left, depth + 1))
            if node.right:
                queue.append((node.right, depth + 1))
```

## Problem: 200. Number of Islands - Medium

```
from collections import deque
class Solution:
    def numIslands(self, grid):
```

```

rows, columns = len(grid), len(grid[0])
islands = 0
for row in range(rows):
    for column in range(columns):
        if grid[row][column] != '1':
            continue
        islands += 1
        grid[row][column] = '0'
        queue = deque([(row, column)])
        while queue:
            current_row, current_column = queue.popleft()
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row = current_row + dr
                next_column = current_column + dc
                if (0 <= next_row < rows and 0 <= next_column < columns
                    and grid[next_row][next_column] == '1'):
                    grid[next_row][next_column] = '0'
                    queue.append((next_row, next_column))
return islands

```

## Problem: 994. Rotting Oranges - Medium

```

from collections import deque
class Solution:
    def orangesRotting(self, grid):
        rows, columns = len(grid), len(grid[0])
        queue = deque()
        fresh = 0
        for row in range(rows):
            for column in range(columns):
                if grid[row][column] == 2:
                    queue.append((row, column))
                elif grid[row][column] == 1:
                    fresh += 1
        minutes = 0
        while queue and fresh:
            for _ in range(len(queue)):
                row, column = queue.popleft()
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    next_row = row + dr
                    next_column = column + dc
                    if (0 <= next_row < rows and 0 <= next_column < columns
                        and grid[next_row][next_column] == 1):
                        grid[next_row][next_column] = 2
                        fresh -= 1
                        queue.append((next_row, next_column))
            minutes += 1
        return -1 if fresh else minutes

```

## Problem: 127. Word Ladder - Hard

```

from collections import deque
class Solution:
    def ladderLength(self, beginWord, endWord, wordList):
        words = set(wordList)
        if endWord not in words:
            return 0
        queue = deque([(beginWord, 1)])
        while queue:
            word, distance = queue.popleft()
            if word == endWord:
                return distance
            for index in range(len(word)):
                for letter in 'abcdefghijklmnopqrstuvwxyz':
                    candidate = word[:index] + letter + word[index + 1:]
                    if candidate in words:
                        words.remove(candidate)
                        queue.append((candidate, distance + 1))

        return 0

```

## Problem: 752. Open the Lock - Medium

```

from collections import deque
class Solution:
    def openLock(self, deadends, target):
        blocked = set(deadends)
        if '0000' in blocked:
            return -1
        queue = deque(['0000', 0])
        visited = {'0000'}
        while queue:
            state, moves = queue.popleft()
            if state == target:
                return moves
            for index in range(4):
                digit = int(state[index])
                for change in (-1, 1):
                    next_digit = (digit + change) % 10
                    candidate = state[:index] + str(next_digit) + state[index +
1:]

                    if candidate not in blocked and candidate not in visited:
                        visited.add(candidate)
                        queue.append((candidate, moves + 1))

        return -1

```

## Problem: 909. Snakes and Ladders - Medium

```

from collections import deque
class Solution:
    def snakesAndLadders(self, board):
        n = len(board)
        def coordinates(square):
            row_from_bottom, column = divmod(square - 1, n)
            row = n - 1 - row_from_bottom
            if row_from_bottom % 2:
                column = n - 1 - column
            return row, column
        queue = deque([(1, 0)])
        visited = {1}
        target = n * n
        while queue:
            square, moves = queue.popleft()
            if square == target:
                return moves
            for next_square in range(square + 1, min(square + 6, target) + 1):
                row, column = coordinates(next_square)
                destination = board[row][column]
                if destination != -1:
                    next_square = destination
                if next_square not in visited:
                    visited.add(next_square)
                    queue.append((next_square, moves + 1))
        return -1

```

## Problem: 542. 01 Matrix - Medium

```

from collections import deque
class Solution:
    def updateMatrix(self, mat):
        rows, columns = len(mat), len(mat[0])
        queue = deque()
        distance = [[-1] * columns for _ in range(rows)]
        for row in range(rows):
            for column in range(columns):
                if mat[row][column] == 0:
                    distance[row][column] = 0
                    queue.append((row, column))
        while queue:
            row, column = queue.popleft()
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_column = row + dr, column + dc
                if (0 <= next_row < rows and 0 <= next_column < columns
                    and distance[next_row][next_column] == -1):
                    distance[next_row][next_column] = distance[row][column] + 1
                    queue.append((next_row, next_column))
        return distance

```

## Problem: 1162. As Far from Land as Possible - Medium

```

from collections import deque
class Solution:
    def maxDistance(self, grid):
        n = len(grid)
        queue = deque()
        for row in range(n):
            for column in range(n):
                if grid[row][column] == 1:
                    queue.append((row, column))
        if not queue or len(queue) == n * n:
            return -1
        distance = -1
        while queue:
            for _ in range(len(queue)):
                row, column = queue.popleft()
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    next_row, next_column = row + dr, column + dc
                    if 0 <= next_row < n and 0 <= next_column < n and
grid[next_row][next_column] == 0:
                        grid[next_row][next_column] = 1
                        queue.append((next_row, next_column))
                distance += 1
        return distance

```

## Problem: 1926. Nearest Exit from Entrance in Maze - Medium

```

from collections import deque
class Solution:
    def nearestExit(self, maze, entrance):
        rows, columns = len(maze), len(maze[0])
        queue = deque([(entrance[0], entrance[1], 0)])
        maze[entrance[0]][entrance[1]] = '+'
        while queue:
            row, column, distance = queue.popleft()
            if distance > 0 and (row in {0, rows - 1} or column in {0, columns -
1}):
                return distance
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_column = row + dr, column + dc
                if (0 <= next_row < rows and 0 <= next_column < columns
                    and maze[next_row][next_column] == '.'):
                    maze[next_row][next_column] = '+'
                    queue.append((next_row, next_column, distance + 1))
        return -1

```

## Problem: 622. Design Circular Queue - Medium

```

class MyCircularQueue:
    def __init__(self, k):
        self.values = [0] * k
        self.capacity = k
        self.front = 0
        self.size = 0
    def enqueue(self, value):
        if self.isFull():
            return False
        index = (self.front + self.size) % self.capacity
        self.values[index] = value
        self.size += 1
        return True
    def dequeue(self):
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True
    def Front(self):
        return -1 if self.isEmpty() else self.values[self.front]
    def Rear(self):
        if self.isEmpty():
            return -1
        return self.values[(self.front + self.size - 1) % self.capacity]
    def isEmpty(self):
        return self.size == 0
    def isFull(self):
        return self.size == self.capacity

```

## Problem: 933. Number of Recent Calls - Easy

```

from collections import deque
class RecentCounter:
    def __init__(self):
        self.requests = deque()
    def ping(self, t):
        self.requests.append(t)
        while self.requests[0] < t - 3000:
            self.requests.popleft()
        return len(self.requests)

```

## Problem: 1091. Shortest Path in Binary Matrix - Medium

```

from collections import deque
class Solution:
    def shortestPathBinaryMatrix(self, grid):

```

```

n = len(grid)
if grid[0][0] or grid[-1][-1]:
    return -1
queue = deque([(0, 0, 1)])
grid[0][0] = 1
directions = ((1, 0), (-1, 0), (0, 1), (0, -1),
              (1, 1), (1, -1), (-1, 1), (-1, -1))
while queue:
    row, column, distance = queue.popleft()
    if row == n - 1 and column == n - 1:
        return distance
    for dr, dc in directions:
        next_row, next_column = row + dr, column + dc
        if (0 <= next_row < n and 0 <= next_column < n
            and grid[next_row][next_column] == 0):
            grid[next_row][next_column] = 1
            queue.append((next_row, next_column, distance + 1))
return -1

```

## Problem: 116. Populating Next Right Pointers in Each Node - Medium

```

from collections import deque
class Solution:
    def connect(self, root):
        if not root:
            return root
        queue = deque([root])
        while queue:
            previous = None
            for _ in range(len(queue)):
                node = queue.popleft()
                if previous:
                    previous.next = node
                previous = node
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            previous.next = None
        return root

```

## Problem: 1670. Design Front Middle Back Queue - Medium

```

from collections import deque
class FrontMiddleBackQueue:
    def __init__(self):
        self.left = deque()
        self.right = deque()
    def _balance(self):

```

```

        while len(self.left) > len(self.right) + 1:
            self.right.appendleft(self.left.pop())
        while len(self.left) < len(self.right):
            self.left.append(self.right.popleft())
    def pushFront(self, val):
        self.left.appendleft(val)
        self._balance()
    def pushMiddle(self, val):
        if len(self.left) > len(self.right):
            self.right.appendleft(self.left.pop())
        self.left.append(val)
    def pushBack(self, val):
        self.right.append(val)
        self._balance()
    def popFront(self):
        if not self.left and not self.right:
            return -1
        value = self.left.popleft() if self.left else self.right.popleft()
        self._balance()
        return value
    def popMiddle(self):
        if not self.left and not self.right:
            return -1
        value = self.left.pop()
        self._balance()
        return value
    def popBack(self):
        if not self.left and not self.right:
            return -1
        value = self.right.pop() if self.right else self.left.pop()
        self._balance()
        return value

```

## Problem: 2258. Escape the Spreading Fire - Hard

```

from collections import deque
class Solution:
    def maximumMinutes(self, grid):
        rows, columns = len(grid), len(grid[0])
        fire_time = [[float('inf')] * columns for _ in range(rows)]
        queue = deque()
        for row in range(rows):
            for column in range(columns):
                if grid[row][column] == 1:
                    fire_time[row][column] = 0
                    queue.append((row, column))
        while queue:
            row, column = queue.popleft()
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_column = row + dr, column + dc
                if (0 <= next_row < rows and 0 <= next_column < columns

```

```

        and grid[next_row][next_column] != 2
        and fire_time[next_row][next_column] == float('inf')):
            fire_time[next_row][next_column] = fire_time[row][column] + 1
            queue.append((next_row, next_column))
def can_escape(wait):
    if wait >= fire_time[0][0]:
        return False
    queue = deque([(0, 0, wait)])
    visited = {(0, 0)}
    while queue:
        row, column, time = queue.popleft()
        if row == rows - 1 and column == columns - 1:
            return time <= fire_time[row][column]
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            next_row, next_column = row + dr, column + dc
            next_time = time + 1
            if not (0 <= next_row < rows and 0 <= next_column < columns):
                continue
            if grid[next_row][next_column] == 2 or (next_row, next_column)
in visited:
                continue
            if next_time >= fire_time[next_row][next_column] and not
(next_row == rows - 1 and next_column == columns - 1):
                continue
            visited.add((next_row, next_column))
            queue.append((next_row, next_column, next_time))
        return False
    low, high = 0, 10**9
    answer = -1
    while low <= high:
        middle = (low + high) // 2
        if can_escape(middle):
            answer = middle
            low = middle + 1
        else:
            high = middle - 1
    return answer

```

## Problem: 232. Implement Queue using Stacks - Easy

```

class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []
    def _move(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
    def push(self, x):
        self.in_stack.append(x)
    def pop(self):

```

```
        self._move()
    return self.out_stack.pop()
def peek(self):
    self._move()
    return self.out_stack[-1]
def empty(self):
    return not self.in_stack and not self.out_stack
```

## Problem: 641. Design Circular Deque - Medium

```
class MyCircularDeque:
    def __init__(self, k):
        self.values = [0] * k
        self.capacity = k
        self.front = 0
        self.size = 0
    def insertFront(self, value):
        if self.isFull():
            return False
        self.front = (self.front - 1) % self.capacity
        self.values[self.front] = value
        self.size += 1
        return True
    def insertLast(self, value):
        if self.isFull():
            return False
        index = (self.front + self.size) % self.capacity
        self.values[index] = value
        self.size += 1
        return True
    def deleteFront(self):
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True
    def deleteLast(self):
        if self.isEmpty():
            return False
        self.size -= 1
        return True
    def getFront(self):
        return -1 if self.isEmpty() else self.values[self.front]
    def getRear(self):
        if self.isEmpty():
            return -1
        return self.values[(self.front + self.size - 1) % self.capacity]
    def isEmpty(self):
        return self.size == 0
```

```
def isFull(self):
    return self.size == self.capacity
```

## Problem: 773. Sliding Puzzle - Hard

```
from collections import deque
class Solution:
    def slidingPuzzle(self, board):
        start = ''.join(str(value) for row in board for value in row)
        target = '123450'
        neighbors = {
            0: (1, 3), 1: (0, 2, 4), 2: (1, 5),
            3: (0, 4), 4: (1, 3, 5), 5: (2, 4)
        }
        queue = deque([(start, start.index('0'), 0)])
        visited = {start}
        while queue:
            state, blank, moves = queue.popleft()
            if state == target:
                return moves
            for next_blank in neighbors[blank]:
                values = list(state)
                values[blank], values[next_blank] = values[next_blank], values[blank]
                candidate = ''.join(values)
                if candidate not in visited:
                    visited.add(candidate)
                    queue.append((candidate, next_blank, moves + 1))
        return -1
```

## Problem: 815. Bus Routes - Hard

```
from collections import defaultdict, deque
class Solution:
    def numBusesToDestination(self, routes, source, target):
        if source == target:
            return 0
        stop_to_routes = defaultdict(list)
        for route_index, route in enumerate(routes):
            for stop in route:
                stop_to_routes[stop].append(route_index)
        queue = deque([(source, 0)])
        visited_stops = {source}
        visited_routes = set()
        while queue:
            stop, buses = queue.popleft()
            for route_index in stop_to_routes[stop]:
                if route_index in visited_routes:
                    continue
```

```

        visited_routes.add(route_index)
    for next_stop in routes[route_index]:
        if next_stop == target:
            return buses + 1
        if next_stop not in visited_stops:
            visited_stops.add(next_stop)
            queue.append((next_stop, buses + 1))

return -1

```

## Problem: 934. Shortest Bridge - Medium

```

from collections import deque
class Solution:
    def shortestBridge(self, grid):
        n = len(grid)
        queue = deque()
        found = False
        def mark(row, column):
            if not (0 <= row < n and 0 <= column < n) or grid[row][column] != 1:
                return
            grid[row][column] = 2
            queue.append((row, column))
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                mark(row + dr, column + dc)
        for row in range(n):
            if found:
                break
            for column in range(n):
                if grid[row][column] == 1:
                    mark(row, column)
                    found = True
                    break
        distance = 0
        while queue:
            for _ in range(len(queue)):
                row, column = queue.popleft()
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    next_row, next_column = row + dr, column + dc
                    if not (0 <= next_row < n and 0 <= next_column < n):
                        continue
                    if grid[next_row][next_column] == 1:
                        return distance
                    if grid[next_row][next_column] == 0:
                        grid[next_row][next_column] = 2
                        queue.append((next_row, next_column))
                distance += 1
        return -1

```

## Problem: 215. Kth Largest Element in an Array - Medium

```
import heapq
class Solution:
    def findKthLargest(self, nums, k):
        heap = []
        for value in nums:
            heapq.heappush(heap, value)
            if len(heap) > k:
                heapq.heappop(heap)
        return heap[0]
```

## Problem: 347. Top K Frequent Elements - Medium

```
import heapq
from collections import Counter
class Solution:
    def topKFrequent(self, nums, k):
        counts = Counter(nums)
        heap = []
        for value, frequency in counts.items():
            heapq.heappush(heap, (frequency, value))
            if len(heap) > k:
                heapq.heappop(heap)
        return [value for frequency, value in heap]
```

## Problem: 692. Top K Frequent Words - Medium

```
from collections import Counter
class Solution:
    def topKFrequent(self, words, k):
        counts = Counter(words)
        ordered = sorted(counts, key=lambda word: (-counts[word], word))
        return ordered[:k]
```

## Problem: 703. Kth Largest Element in a Stream - Easy

```
import heapq
class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for value in nums:
            self.add(value)
    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
```

```
        heapq.heappop(self.heap)
    return self.heap[0]
```

## Problem: 973. K Closest Points to Origin - Medium

```
import heapq
class Solution:
    def kClosest(self, points, k):
        heap = []
        for x, y in points:
            distance = x * x + y * y
            heapq.heappush(heap, (-distance, x, y))
            if len(heap) > k:
                heapq.heappop(heap)
        return [[x, y] for distance, x, y in heap]
```

## Problem: 1046. Last Stone Weight - Easy

```
import heapq
class Solution:
    def lastStoneWeight(self, stones):
        heap = [-stone for stone in stones]
        heapq.heapify(heap)
        while len(heap) > 1:
            heaviest = -heapq.heappop(heap)
            second = -heapq.heappop(heap)
            if heaviest != second:
                heapq.heappush(heap, -(heaviest - second))
        return -heap[0] if heap else 0
```

## Problem: 378. Kth Smallest Element in a Sorted Matrix - Medium

```
import heapq
class Solution:
    def kthSmallest(self, matrix, k):
        heap = []
        for row in range(min(k, len(matrix))):
            heapq.heappush(heap, (matrix[row][0], row, 0))
        value = 0
        for _ in range(k):
            value, row, column = heapq.heappop(heap)
            if column + 1 < len(matrix[row]):
                heapq.heappush(heap, (matrix[row][column + 1], row, column + 1))
        return value
```

## Problem: 23. Merge k Sorted Lists - Hard

```
import heapq
class Solution:
    def mergeKLists(self, lists):
        heap = []
        for index, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, index, node))
        dummy = ListNode(0)
        tail = dummy
        while heap:
            value, index, node = heapq.heappop(heap)
            tail.next = node
            tail = node
            if node.next:
                heapq.heappush(heap, (node.next.val, index, node.next))
        return dummy.next
```

## Problem: 373. Find K Pairs with Smallest Sums - Medium

```
import heapq
class Solution:
    def kSmallestPairs(self, nums1, nums2, k):
        if not nums1 or not nums2:
            return []
        heap = [(nums1[i] + nums2[0], i, 0) for i in range(min(k, len(nums1)))]
        heapq.heapify(heap)
        result = []
        while heap and len(result) < k:
            total, i, j = heapq.heappop(heap)
            result.append([nums1[i], nums2[j]])
            if j + 1 < len(nums2):
                heapq.heappush(heap, (nums1[i] + nums2[j + 1], i, j + 1))
        return result
```

## Problem: 632. Smallest Range Covering Elements from K Lists - Hard

```
import heapq
class Solution:
    def smallestRange(self, nums):
        heap = []
        current_max = float('-inf')
        for list_index, values in enumerate(nums):
            heapq.heappush(heap, (values[0], list_index, 0))
            current_max = max(current_max, values[0])
        best = [heap[0][0], current_max]
```

```

while True:
    current_min, list_index, value_index = heapq.heappop(heap)
    if current_max - current_min < best[1] - best[0]:
        best = [current_min, current_max]
    if value_index + 1 == len(nums[list_index]):
        break
    next_value = nums[list_index][value_index + 1]
    current_max = max(current_max, next_value)
    heapq.heappush(heap, (next_value, list_index, value_index + 1))
return best

```

## Problem: 295. Find Median from Data Stream - Hard

```

import heapq
class MedianFinder:
    def __init__(self):
        self.lower = []
        self.upper = []
    def addNum(self, num):
        heapq.heappush(self.lower, -num)
        heapq.heappush(self.upper, -heapq.heappop(self.lower))
        if len(self.upper) > len(self.lower):
            heapq.heappush(self.lower, -heapq.heappop(self.upper))
    def findMedian(self):
        if len(self.lower) > len(self.upper):
            return -self.lower[0]
        return (-self.lower[0] + self.upper[0]) / 2

```

## Problem: 502. IPO - Hard

```

import heapq
class Solution:
    def findMaximizedCapital(self, k, w, profits, capital):
        projects = sorted(zip(capital, profits))
        available = []
        index = 0
        for _ in range(k):
            while index < len(projects) and projects[index][0] <= w:
                heapq.heappush(available, -projects[index][1])
                index += 1
            if not available:
                break
            w -= heapq.heappop(available)
        return w

```

## Problem: 253. Meeting Rooms II - Medium

```
import heapq
class Solution:
    def minMeetingRooms(self, intervals):
        intervals.sort()
        ending_times = []
        for start, end in intervals:
            if ending_times and ending_times[0] <= start:
                heapq.heappop(ending_times)
            heapq.heappush(ending_times, end)
        return len(ending_times)
```

## Problem: 621. Task Scheduler - Medium

```
import heapq
from collections import Counter
class Solution:
    def leastInterval(self, tasks, n):
        counts = Counter(tasks)
        heap = [-frequency for frequency in counts.values()]
        heapq.heapify(heap)
        time = 0
        while heap:
            waiting = []
            for _ in range(n + 1):
                time += 1
                if heap:
                    remaining = -heapq.heappop(heap) - 1
                    if remaining:
                        waiting.append(remaining)
                if not heap and not waiting:
                    break
            for remaining in waiting:
                heapq.heappush(heap, -remaining)
        return time
```

## Problem: 630. Course Schedule III - Hard

```
import heapq
class Solution:
    def scheduleCourse(self, courses):
        courses.sort(key=lambda course: course[1])
        selected = []
        elapsed = 0
        for duration, deadline in courses:
            heapq.heappush(selected, -duration)
            elapsed += duration
            if elapsed > deadline:
```

```

        elapsed += heapq.heappop(selected)
    return len(selected)

```

## Problem: 1834. Single-Threaded CPU - Medium

```

import heapq
class Solution:
    def getOrder(self, tasks):
        ordered = sorted((start, duration, index) for index, (start, duration) in
            enumerate(tasks))
        available = []
        result = []
        time = 0
        index = 0
        while index < len(ordered) or available:
            if not available and time < ordered[index][0]:
                time = ordered[index][0]
            while index < len(ordered) and ordered[index][0] <= time:
                start, duration, task_index = ordered[index]
                heapq.heappush(available, (duration, task_index))
                index += 1
            duration, task_index = heapq.heappop(available)
            time += duration
            result.append(task_index)
        return result

```

## Problem: 2402. Meeting Rooms III - Hard

```

import heapq
class Solution:
    def mostBooked(self, n, meetings):
        meetings.sort()
        available = list(range(n))
        occupied = []
        used = [0] * n
        for start, end in meetings:
            duration = end - start
            while occupied and occupied[0][0] <= start:
                finish, room = heapq.heappop(occupied)
                heapq.heappush(available, room)
            if available:
                room = heapq.heappop(available)
                finish = end
            else:
                finish, room = heapq.heappop(occupied)
                finish += duration
            heapq.heappush(occupied, (finish, room))

```

```

        used[room] += 1
    return max(range(n), key=lambda room: used[room])

```

## Problem: 1642. Furthest Building You Can Reach - Medium

```

import heapq
class Solution:
    def furthestBuilding(self, heights, bricks, ladders):
        climbs = []
        for index in range(len(heights) - 1):
            climb = heights[index + 1] - heights[index]
            if climb <= 0:
                continue
            heapq.heappush(climbs, climb)
            if len(climbs) > ladders:
                bricks -= heapq.heappop(climbs)
            if bricks < 0:
                return index
        return len(heights) - 1

```

## Problem: 871. Minimum Number of Refueling Stops - Hard

```

import heapq
class Solution:
    def minRefuelStops(self, target, startFuel, stations):
        stations = stations + [[target, 0]]
        fuel_heap = []
        previous = 0
        fuel = startFuel
        stops = 0
        for position, amount in stations:
            fuel -= position - previous
            while fuel < 0 and fuel_heap:
                fuel += -heapq.heappop(fuel_heap)
                stops += 1
            if fuel < 0:
                return -1
            heapq.heappush(fuel_heap, -amount)
            previous = position
        return stops

```

## Problem: 857. Minimum Cost to Hire K Workers - Hard

```

import heapq
class Solution:
    def mincostToHireWorkers(self, quality, wage, k):

```

```

workers = sorted((w / q, q) for q, w in zip(quality, wage))
qualities = []
quality_sum = 0
best = float('inf')
for ratio, worker_quality in workers:
    heapq.heappush(qualities, -worker_quality)
    quality_sum += worker_quality
    if len(qualities) > k:
        quality_sum += heapq.heappop(qualities)
    if len(qualities) == k:
        best = min(best, quality_sum * ratio)
return best

```

## Problem: 1383. Maximum Performance of a Team - Hard

```

import heapq
class Solution:
    def maxPerformance(self, n, speed, efficiency, k):
        engineers = sorted(zip(efficiency, speed), reverse=True)
        speeds = []
        speed_sum = 0
        best = 0
        modulo = 10**9 + 7
        for current_efficiency, current_speed in engineers:
            heapq.heappush(speeds, current_speed)
            speed_sum += current_speed
            if len(speeds) > k:
                speed_sum -= heapq.heappop(speeds)
            best = max(best, speed_sum * current_efficiency)
        return best % modulo

```

## Problem: 1094. Car Pooling - Medium

```

class Solution:
    def carPooling(self, trips, capacity):
        difference = [0] * 1001
        for passengers, start, end in trips:
            difference[start] += passengers
            difference[end] -= passengers
        current = 0
        for change in difference:
            current += change
            if current > capacity:
                return False
        return True

```

## Problem: 1825. Finding MK Average - Hard

```

from bisect import bisect_left, insert
from collections import deque
class MKAverage:
    def __init__(self, m, k):
        self.m = m
        self.k = k
        self.values = deque()
        self.sorted_values = []
    def addElement(self, num):
        self.values.append(num)
        insert(self.sorted_values, num)
        if len(self.values) > self.m:
            outgoing = self.values.popleft()
            self.sorted_values.pop(bisect_left(self.sorted_values, outgoing))
    def calculateMKAverage(self):
        if len(self.values) < self.m:
            return -1
        middle = self.sorted_values[self.k:self.m - self.k]
        return sum(middle) // len(middle)

```

## Problem: 2532. Time to Cross a Bridge - Hard

```

import heapq
class Solution:
    def findCrossingTime(self, n, k, time):
        waiting_left = [(-time[i][0] - time[i][2], -i) for i in range(k)]
        waiting_right = []
        working_left = []
        working_right = []
        heapq.heapify(waiting_left)
        current = 0
        while n > 0 or working_left or working_right or waiting_right:
            while working_left and working_left[0][0] <= current:
                finish, worker = heapq.heappop(working_left)
                heapq.heappush(waiting_left, (-time[worker][0] - time[worker][2],
-worker))
            while working_right and working_right[0][0] <= current:
                finish, worker = heapq.heappop(working_right)
                heapq.heappush(waiting_right, (-time[worker][0] - time[worker][2],
-worker))
            if waiting_right:
                _, negative_worker = heapq.heappop(waiting_right)
                worker = -negative_worker
                current += time[worker][2]
                heapq.heappush(working_left, (current + time[worker][3], worker))
            elif n > 0 and waiting_left:
                _, negative_worker = heapq.heappop(waiting_left)
                worker = -negative_worker
                current += time[worker][0]
                n -= 1

```

```

        heapq.heappush(working_right, (current + time[worker][1], worker))
    else:
        next_time = []
        if working_left:
            next_time.append(working_left[0][0])
        if working_right:
            next_time.append(working_right[0][0])
        current = min(next_time)
    return current

```

## Problem: 480. Sliding Window Median - Hard

```

from bisect import bisect_left, insort
class Solution:
    def medianSlidingWindow(self, nums, k):
        window = sorted(nums[:k])
        result = []
        def median():
            if k % 2:
                return float(window[k // 2])
            return (window[k // 2 - 1] + window[k // 2]) / 2
        result.append(median())
        for right in range(k, len(nums)):
            window.pop(bisect_left(window, nums[right - k]))
            insort(window, nums[right])
            result.append(median())
        return result

```

## Problem: 786. K-th Smallest Prime Fraction - Medium

```

import heapq
class Solution:
    def kthSmallestPrimeFraction(self, arr, k):
        heap = [(arr[i] / arr[-1], i, len(arr) - 1) for i in range(len(arr) - 1)]
        heapq.heapify(heap)
        for _ in range(k - 1):
            fraction, numerator, denominator = heapq.heappop(heap)
            if denominator - 1 > numerator:
                next_denominator = denominator - 1
                heapq.heappush(heap, (arr[numerator] / arr[next_denominator],
numerator, next_denominator))
            _, numerator, denominator = heapq.heappop(heap)
        return [arr[numerator], arr[denominator]]

```

## Problem: 643. Maximum Average Subarray I - Easy

```
class Solution:
    def findMaxAverage(self, nums, k):
        window_sum = sum(nums[:k])
        best_sum = window_sum
        for right in range(k, len(nums)):
            window_sum += nums[right] - nums[right - k]
            best_sum = max(best_sum, window_sum)
        return best_sum / k
```

## Problem: 1456. Maximum Number of Vowels in a Substring of Given Length - Medium

```
class Solution:
    def maxVowels(self, s, k):
        vowels = set('aeiou')
        current = sum(character in vowels for character in s[:k])
        best = current
        for right in range(k, len(s)):
            if s[right - k] in vowels:
                current -= 1
            if s[right] in vowels:
                current += 1
            best = max(best, current)
        return best
```

## Problem: 1343. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold - Medium

```
class Solution:
    def numOfSubarrays(self, arr, k, threshold):
        target_sum = k * threshold
        window_sum = sum(arr[:k])
        answer = int(window_sum >= target_sum)
        for right in range(k, len(arr)):
            window_sum += arr[right] - arr[right - k]
            if window_sum >= target_sum:
                answer += 1
        return answer
```

## Problem: 3. Longest Substring Without Repeating Characters - Medium

```
class Solution:
    def lengthOfLongestSubstring(self, s):
        last_seen = {}
        left = 0
```

```
best = 0
for right, character in enumerate(s):
    if character in last_seen and last_seen[character] >= left:
        left = last_seen[character] + 1
    last_seen[character] = right
    best = max(best, right - left + 1)
return best
```

## Problem: 904. Fruit Into Baskets - Medium

```
class Solution:
    def totalFruit(self, fruits):
        counts = {}
        left = 0
        best = 0
        for right, fruit in enumerate(fruits):
            counts[fruit] = counts.get(fruit, 0) + 1
            while len(counts) > 2:
                outgoing = fruits[left]
                counts[outgoing] -= 1
                if counts[outgoing] == 0:
                    del counts[outgoing]
                left += 1
            best = max(best, right - left + 1)
        return best
```

## Problem: 1004. Max Consecutive Ones III - Medium

```
class Solution:
    def longestOnes(self, nums, k):
        left = 0
        zeros = 0
        best = 0
        for right, value in enumerate(nums):
            if value == 0:
                zeros += 1
            while zeros > k:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            best = max(best, right - left + 1)
        return best
```

## Problem: 209. Minimum Size Subarray Sum - Medium

```
class Solution:
    def minSubArrayLen(self, target, nums):
        left = 0
        current_sum = 0
        best = float('inf')
        for right, value in enumerate(nums):
            current_sum += value
            while current_sum >= target:
                best = min(best, right - left + 1)
                current_sum -= nums[left]
                left += 1
        return 0 if best == float('inf') else best
```

## Problem: 76. Minimum Window Substring - Hard

```
from collections import Counter
class Solution:
    def minWindow(self, s, t):
        need = Counter(t)
        missing = len(t)
        left = 0
        best_start = 0
        best_length = float('inf')
        for right, character in enumerate(s):
            if need[character] > 0:
                missing -= 1
                need[character] -= 1
            while missing == 0:
                if right - left + 1 < best_length:
                    best_start = left
                    best_length = right - left + 1
                outgoing = s[left]
                need[outgoing] += 1
                if need[outgoing] > 0:
                    missing += 1
                left += 1
        return '' if best_length == float('inf') else s[best_start:best_start + best_length]
```

## Problem: 713. Subarray Product Less Than K - Medium

```
class Solution:
    def numSubarrayProductLessThanK(self, nums, k):
        if k <= 1:
            return 0
        left = 0
        product = 1
```

```
answer = 0
for right, value in enumerate(nums):
    product *= value
    while product >= k:
        product //= nums[left]
        left += 1
    answer += right - left + 1
return answer
```

## Problem: 438. Find All Anagrams in a String - Medium

```
from collections import Counter
class Solution:
    def findAnagrams(self, s, p):
        if len(p) > len(s):
            return []
        need = Counter(p)
        window = Counter(s[:len(p)])
        result = []
        if window == need:
            result.append(0)
        for right in range(len(p), len(s)):
            window[s[right]] += 1
            outgoing = s[right - len(p)]
            window[outgoing] -= 1
            if window[outgoing] == 0:
                del window[outgoing]
            if window == need:
                result.append(right - len(p) + 1)
        return result
```

## Problem: 992. Subarrays with K Different Integers - Hard

```
class Solution:
    def subarraysWithKDistinct(self, nums, k):
        def at_most(limit):
            counts = {}
            left = 0
            answer = 0
            for right, value in enumerate(nums):
                counts[value] = counts.get(value, 0) + 1
                while len(counts) > limit:
                    outgoing = nums[left]
                    counts[outgoing] -= 1
                    if counts[outgoing] == 0:
                        del counts[outgoing]
                    left += 1
                answer += right - left + 1
            return answer
```

```
        return answer
    return at_most(k) - at_most(k - 1)
```

## Problem: 239. Sliding Window Maximum - Hard

```
from collections import deque
class Solution:
    def maxSlidingWindow(self, nums, k):
        window = deque()
        result = []
        for right, value in enumerate(nums):
            while window and window[0] <= right - k:
                window.popleft()
            while window and nums[window[-1]] <= value:
                window.pop()
            window.append(right)
            if right >= k - 1:
                result.append(nums[window[0]])
        return result
```

## Problem: 1423. Maximum Points You Can Obtain from Cards - Medium

```
class Solution:
    def maxScore(self, cardPoints, k):
        total = sum(cardPoints)
        window_size = len(cardPoints) - k
        if window_size == 0:
            return total
        window_sum = sum(cardPoints[:window_size])
        smallest_middle = window_sum
        for right in range(window_size, len(cardPoints)):
            window_sum += cardPoints[right] - cardPoints[right - window_size]
            smallest_middle = min(smallest_middle, window_sum)
        return total - smallest_middle
```

## Problem: 340. Longest Substring with At Most K Distinct Characters - Medium

```
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s, k):
        counts = {}
        left = 0
        best = 0
        for right, character in enumerate(s):
            counts[character] = counts.get(character, 0) + 1
            while len(counts) > k:
```

```

        outgoing = s[left]
        counts[outgoing] -= 1
        if counts[outgoing] == 0:
            del counts[outgoing]
        left += 1
        best = max(best, right - left + 1)
    return best

```

## Problem: 1052. Grumpy Bookstore Owner - Medium

```

class Solution:
    def maxSatisfied(self, customers, grumpy, minutes):
        baseline = sum(customers[i] for i in range(len(customers)) if grumpy[i] == 0)

        recovered = sum(customers[i] for i in range(minutes) if grumpy[i] == 1)
        best_recovered = recovered
        for right in range(minutes, len(customers)):
            if grumpy[right] == 1:
                recovered += customers[right]
            if grumpy[right - minutes] == 1:
                recovered -= customers[right - minutes]
            best_recovered = max(best_recovered, recovered)
        return baseline + best_recovered

```

## Problem: 1234. Replace the Substring for Balanced String - Medium

```

from collections import Counter
class Solution:
    def balancedString(self, s):
        required = len(s) // 4
        counts = Counter(s)
        left = 0
        best = len(s)
        for right, character in enumerate(s):
            counts[character] -= 1
            while left <= right and all(counts[c] <= required for c in 'QWER'):
                best = min(best, right - left + 1)
                counts[s[left]] += 1
                left += 1
        return best

```

## Problem: 1248. Count Number of Nice Subarrays - Medium

```

class Solution:
    def numberOfSubarrays(self, nums, k):
        def at_most(limit):

```

```
left = 0
answer = 0
for right, value in enumerate(nums):
    limit -= value % 2
    while limit < 0:
        limit += nums[left] % 2
        left += 1
    answer += right - left + 1
return answer
return at_most(k) - at_most(k - 1)
```

## Problem: 1358. Number of Substrings Containing All Three Characters - Medium

```
class Solution:
    def numberOfSubstrings(self, s):
        last_seen = [-1, -1, -1]
        answer = 0
        for right, character in enumerate(s):
            last_seen[ord(character) - ord('a')] = right
            answer += min(last_seen) + 1
        return answer
```

## Problem: 1493. Longest Subarray of 1's After Deleting One Element - Medium

```
class Solution:
    def longestSubarray(self, nums):
        left = 0
        zeros = 0
        best = 0
        for right, value in enumerate(nums):
            zeros += value == 0
            while zeros > 1:
                zeros -= nums[left] == 0
                left += 1
            best = max(best, right - left)
        return best
```

## Problem: 1658. Minimum Operations to Reduce X to Zero - Medium

```
class Solution:
    def minOperations(self, nums, x):
        target = sum(nums) - x
        if target < 0:
            return -1
```

```
if target == 0:
    return len(nums)
left = 0
current = 0
longest = -1
for right, value in enumerate(nums):
    current += value
    while current > target:
        current -= nums[left]
        left += 1
    if current == target:
        longest = max(longest, right - left + 1)
return -1 if longest == -1 else len(nums) - longest
```

## Problem: 1696. Jump Game VI - Medium

```
from collections import deque
class Solution:
    def maxResult(self, nums, k):
        scores = [0] * len(nums)
        scores[0] = nums[0]
        candidates = deque([0])
        for i in range(1, len(nums)):
            while candidates and candidates[0] < i - k:
                candidates.popleft()
            scores[i] = scores[candidates[0]] + nums[i]
            while candidates and scores[candidates[-1]] <= scores[i]:
                candidates.pop()
            candidates.append(i)
        return scores[-1]
```

## Problem: 2024. Maximize the Confusion of an Exam - Medium

```
class Solution:
    def maxConsecutiveAnswers(self, answerKey, k):
        def longest(target):
            left = 0
            changes = 0
            best = 0
            for right, answer in enumerate(answerKey):
                changes += answer != target
                while changes > k:
                    changes -= answerKey[left] != target
                    left += 1
                best = max(best, right - left + 1)
            return best
        return max(longest('T'), longest('F'))
```

## Problem: 2461. Maximum Sum of Distinct Subarrays With Length K - Medium

```
class Solution:
    def maximumSubarraySum(self, nums, k):
        counts = {}
        window_sum = 0
        best = 0
        for right, value in enumerate(nums):
            counts[value] = counts.get(value, 0) + 1
            window_sum += value
            if right >= k:
                outgoing = nums[right - k]
                window_sum -= outgoing
                counts[outgoing] -= 1
                if counts[outgoing] == 0:
                    del counts[outgoing]
            if right >= k - 1 and len(counts) == k:
                best = max(best, window_sum)
        return best
```

## Problem: 2958. Length of Longest Subarray With at Most K Frequency - Medium

```
class Solution:
    def maxSubarrayLength(self, nums, k):
        counts = {}
        left = 0
        best = 0
        for right, value in enumerate(nums):
            counts[value] = counts.get(value, 0) + 1
            while counts[value] > k:
                outgoing = nums[left]
                counts[outgoing] -= 1
                left += 1
            best = max(best, right - left + 1)
        return best
```

## Problem: 567. Permutation in String - Medium

```
from collections import Counter
class Solution:
    def checkInclusion(self, s1, s2):
        if len(s1) > len(s2):
            return False
        need = Counter(s1)
        window = Counter()
```

```

matched = 0
left = 0
for right, character in enumerate(s2):
    window[character] = window.get(character, 0) + 1
    if window[character] <= need[character]:
        matched += 1
    if right - left + 1 > len(s1):
        outgoing = s2[left]
        if window[outgoing] <= need[outgoing]:
            matched -= 1
        window[outgoing] -= 1
        left += 1
    if matched == len(s1):
        return True
return False

```

## Problem: 862. Shortest Subarray with Sum at Least K - Hard

```

from collections import deque
class Solution:
    def shortestSubarray(self, nums, k):
        prefix = [0]
        for value in nums:
            prefix.append(prefix[-1] + value)
        candidates = deque()
        best = len(nums) + 1
        for right, total in enumerate(prefix):
            while candidates and total - prefix[candidates[0]] >= k:
                best = min(best, right - candidates.popleft())
            while candidates and prefix[candidates[-1]] >= total:
                candidates.pop()
            candidates.append(right)
        return -1 if best == len(nums) + 1 else best

```

## Problem: 930. Binary Subarrays With Sum - Medium

```

class Solution:
    def numSubarraysWithSum(self, nums, goal):
        def at_most(limit):
            if limit < 0:
                return 0
            left = 0
            answer = 0
            for right, value in enumerate(nums):
                limit -= value
                while limit < 0:
                    limit += nums[left]
                    left += 1
                answer += right - left + 1

```

```
        return answer
    return at_most(goal) - at_most(goal - 1)
```

## Problem: 1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit - Medium

```
from collections import deque
class Solution:
    def longestSubarray(self, nums, limit):
        minimums = deque()
        maximums = deque()
        left = 0
        best = 0
        for right, value in enumerate(nums):
            while minimums and nums[minimums[-1]] >= value:
                minimums.pop()
            while maximums and nums[maximums[-1]] <= value:
                maximums.pop()
            minimums.append(right)
            maximums.append(right)
            while nums[maximums[0]] - nums[minimums[0]] > limit:
                if minimums[0] == left:
                    minimums.popleft()
                if maximums[0] == left:
                    maximums.popleft()
                left += 1
            best = max(best, right - left + 1)
        return best
```

## Problem: 30. Substring with Concatenation of All Words - Hard

```
from collections import Counter
class Solution:
    def findSubstring(self, s, words):
        if not s or not words:
            return []
        word_length = len(words[0])
        word_count = len(words)
        total_length = word_length * word_count
        required = Counter(words)
        result = []
        for offset in range(word_length):
            left = offset
            used = Counter()
            matched = 0
            for right in range(offset, len(s) - word_length + 1, word_length):
                word = s[right:right + word_length]
                if word not in required:
                    left = right + word_length
                    continue
                used[word] += 1
                matched += 1
                if matched == word_count:
                    result.append(s[left:right + word_length])
                    left = right + word_length
```

```

        used.clear()
        matched = 0
        continue
    used[word] += 1
    matched += 1
    while used[word] > required[word]:
        outgoing = s[left:left + word_length]
        used[outgoing] -= 1
        left += word_length
        matched -= 1
    if matched == word_count:
        result.append(left)
        outgoing = s[left:left + word_length]
        used[outgoing] -= 1
        left += word_length
        matched -= 1
    return result

```

## Problem: 159. Longest Substring with At Most Two Distinct Characters - Medium

```

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s):
        counts = {}
        left = 0
        best = 0
        for right, character in enumerate(s):
            counts[character] = counts.get(character, 0) + 1
            while len(counts) > 2:
                outgoing = s[left]
                counts[outgoing] -= 1
                if counts[outgoing] == 0:
                    del counts[outgoing]
                left += 1
            best = max(best, right - left + 1)
        return best

```

## Problem: 187. Repeated DNA Sequences - Medium

```

class Solution:
    def findRepeatedDnaSequences(self, s):
        seen = set()
        repeated = set()
        for start in range(len(s) - 9):
            sequence = s[start:start + 10]
            if sequence in seen:
                repeated.add(sequence)
            else:

```

```
        seen.add(sequence)
    return list(repeated)
```

## Problem: 219. Contains Duplicate II - Easy

```
class Solution:
    def containsNearbyDuplicate(self, nums, k):
        last_seen = {}
        for index, value in enumerate(nums):
            if value in last_seen and index - last_seen[value] <= k:
                return True
            last_seen[value] = index
        return False
```

## Problem: 220. Contains Duplicate III - Hard

```
class Solution:
    def containsNearbyAlmostDuplicate(self, nums, indexDiff, valueDiff):
        if valueDiff < 0:
            return False
        width = valueDiff + 1
        buckets = {}
        def bucket_id(value):
            return value // width if value >= 0 else -((-value - 1) // width) - 1
        for index, value in enumerate(nums):
            bucket = bucket_id(value)
            if bucket in buckets:
                return True
            if bucket - 1 in buckets and abs(value - buckets[bucket - 1]) <= valueDiff:
                return True
            if bucket + 1 in buckets and abs(value - buckets[bucket + 1]) <= valueDiff:
                return True
            buckets[bucket] = value
            if index >= indexDiff:
                del buckets[bucket_id(nums[index - indexDiff])]
        return False
```

## Problem: 395. Longest Substring with At Least K Repeating Characters - Medium

```
from collections import Counter
class Solution:
    def longestSubstring(self, s, k):
        if len(s) < k:
```

```

        return 0
    counts = Counter(s)
    for index, character in enumerate(s):
        if counts[character] < k:
            return max(self.longestSubstring(s[:index], k),
self.longestSubstring(s[index + 1:], k))
    return len(s)

```

## Problem: 413. Arithmetic Slices - Medium

```

class Solution:
    def numberOfArithmeticSlices(self, nums):
        current = 0
        answer = 0
        for index in range(2, len(nums)):
            if nums[index] - nums[index - 1] == nums[index - 1] - nums[index - 2]:
                current += 1
                answer += current
            else:
                current = 0
        return answer

```

## Problem: 424. Longest Repeating Character Replacement - Medium

```

class Solution:
    def characterReplacement(self, s, k):
        counts = {}
        left = 0
        most_frequent = 0
        best = 0
        for right, character in enumerate(s):
            counts[character] = counts.get(character, 0) + 1
            most_frequent = max(most_frequent, counts[character])
            while right - left + 1 - most_frequent > k:
                counts[s[left]] -= 1
                left += 1
            best = max(best, right - left + 1)
        return best

```

## Problem: 480. Sliding Window Median - Hard

```

from bisect import bisect_left, insort
class Solution:
    def medianSlidingWindow(self, nums, k):
        window = sorted(nums[:k])
        result = []

```

```

def median():
    if k % 2:
        return float(window[k // 2])
    return (window[k // 2 - 1] + window[k // 2]) / 2
result.append(median())
for right in range(k, len(nums)):
    window.pop(bisect_left(window, nums[right - k]))
    insort(window, nums[right])
    result.append(median())
return result

```

## Problem: 487. Max Consecutive Ones II - Medium

```

class Solution:
    def findMaxConsecutiveOnes(self, nums):
        left = 0
        zeros = 0
        best = 0
        for right, value in enumerate(nums):
            zeros += value == 0
            while zeros > 1:
                zeros -= nums[left] == 0
                left += 1
            best = max(best, right - left + 1)
        return best

```

## Problem: 20. Valid Parentheses - Easy

```

class Solution:
    def isValid(self, s):
        pairs = {'(': ')', '[': ']', '{': '}'}
        stack = []
        for character in s:
            if character in pairs:
                if not stack or stack.pop() != pairs[character]:
                    return False
            else:
                stack.append(character)
        return not stack

```

## Problem: 32. Longest Valid Parentheses - Hard

```

class Solution:
    def longestValidParentheses(self, s):
        stack = [-1]
        best = 0

```

```
for index, character in enumerate(s):
    if character == '(':
        stack.append(index)
    else:
        stack.pop()
        if not stack:
            stack.append(index)
        else:
            best = max(best, index - stack[-1])
return best
```

## Problem: 678. Valid Parenthesis String - Medium

```
class Solution:
    def checkValidString(self, s):
        minimum_open = 0
        maximum_open = 0
        for character in s:
            if character == '(':
                minimum_open += 1
                maximum_open += 1
            elif character == ')':
                minimum_open = max(0, minimum_open - 1)
                maximum_open -= 1
            else:
                minimum_open = max(0, minimum_open - 1)
                maximum_open += 1
            if maximum_open < 0:
                return False
        return minimum_open == 0
```

## Problem: 921. Minimum Add to Make Parentheses Valid - Medium

```
class Solution:
    def minAddToMakeValid(self, s):
        open_count = 0
        additions = 0
        for character in s:
            if character == '(':
                open_count += 1
            elif open_count:
                open_count -= 1
            else:
                additions += 1
        return additions + open_count
```

## Problem: 1541. Minimum Insertions to Balance a Parentheses String - Medium

```
class Solution:
    def minInsertions(self, s):
        open_count = 0
        insertions = 0
        index = 0
        while index < len(s):
            if s[index] == '(':
                open_count += 1
            else:
                if index + 1 < len(s) and s[index + 1] == ')':
                    index += 1
                else:
                    insertions += 1
                if open_count:
                    open_count -= 1
                else:
                    insertions += 1
            index += 1
        return insertions + 2 * open_count
```

## Problem: 496. Next Greater Element I - Easy

```
class Solution:
    def nextGreaterElement(self, nums1, nums2):
        next_greater = {}
        stack = []
        for value in nums2:
            while stack and stack[-1] < value:
                next_greater[stack.pop()] = value
            stack.append(value)
        return [next_greater.get(value, -1) for value in nums1]
```

## Problem: 503. Next Greater Element II - Medium

```
class Solution:
    def nextGreaterElements(self, nums):
        result = [-1] * len(nums)
        stack = []
        for index in range(2 * len(nums)):
            current_index = index % len(nums)
            while stack and nums[stack[-1]] < nums[current_index]:
                result[stack.pop()] = nums[current_index]
            if index < len(nums):
                stack.append(index)
```

```
        stack.append(current_index)
    return result
```

## Problem: 739. Daily Temperatures - Medium

```
class Solution:
    def dailyTemperatures(self, temperatures):
        answer = [0] * len(temperatures)
        stack = []
        for today, temperature in enumerate(temperatures):
            while stack and temperatures[stack[-1]] < temperature:
                previous = stack.pop()
                answer[previous] = today - previous
            stack.append(today)
        return answer
```

## Problem: 901. Online Stock Span - Medium

```
class StockSpanner:
    def __init__(self):
        self.stack = []
    def next(self, price):
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span
```

## Problem: 84. Largest Rectangle in Histogram - Hard

```
class Solution:
    def largestRectangleArea(self, heights):
        stack = []
        best = 0
        extended = heights + [0]
        for right, height in enumerate(extended):
            while stack and extended[stack[-1]] > height:
                bar = stack.pop()
                left = stack[-1] + 1 if stack else 0
                width = right - left
                best = max(best, extended[bar] * width)
            stack.append(right)
        return best
```

## Problem: 85. Maximal Rectangle - Hard

```

class Solution:
    def maximalRectangle(self, matrix):
        if not matrix:
            return 0
        heights = [0] * len(matrix[0])
        best = 0
        for row in matrix:
            for column, value in enumerate(row):
                heights[column] = heights[column] + 1 if value == '1' else 0
                best = max(best, self.largestRectangleArea(heights))
        return best
    def largestRectangleArea(self, heights):
        stack = []
        best = 0
        for right in range(len(heights) + 1):
            height = heights[right] if right < len(heights) else 0
            while stack and heights[stack[-1]] > height:
                bar = stack.pop()
                left = stack[-1] + 1 if stack else 0
                best = max(best, heights[bar] * (right - left))
            stack.append(right)
        return best

```

## Problem: 907. Sum of Subarray Minimums - Medium

```

class Solution:
    def sumSubarrayMins(self, arr):
        modulo = 10**9 + 7
        stack = []
        answer = 0
        for index in range(len(arr) + 1):
            current = arr[index] if index < len(arr) else 0
            while stack and arr[stack[-1]] > current:
                minimum_index = stack.pop()
                left_count = minimum_index - (stack[-1] if stack else -1)
                right_count = index - minimum_index
                answer += arr[minimum_index] * left_count * right_count
            stack.append(index)
        return answer % modulo

```

## Problem: 150. Evaluate Reverse Polish Notation - Medium

```

class Solution:
    def evalRPN(self, tokens):
        stack = []
        for token in tokens:
            if token not in {'+', '-', '*', '/'}:

```

```
        stack.append(int(token))
        continue
    right = stack.pop()
    left = stack.pop()
    if token == '+':
        stack.append(left + right)
    elif token == '-':
        stack.append(left - right)
    elif token == '*':
        stack.append(left * right)
    else:
        stack.append(int(left / right))
    return stack[-1]
```

## Problem: 224. Basic Calculator - Hard

```
class Solution:
    def calculate(self, s):
        stack = []
        total = 0
        number = 0
        sign = 1
        for character in s:
            if character.isdigit():
                number = number * 10 + int(character)
            elif character in '+-':
                total += sign * number
                number = 0
                sign = 1 if character == '+' else -1
            elif character == '(':
                stack.append(total)
                stack.append(sign)
                total = 0
                sign = 1
            elif character == ')':
                total += sign * number
                number = 0
                total *= stack.pop()
                total += stack.pop()
        return total + sign * number
```

## Problem: 227. Basic Calculator II - Medium

```
class Solution:
    def calculate(self, s):
        stack = []
        number = 0
        operator = '+'
        for index, character in enumerate(s + '+'):
```

```
if character.isdigit():
    number = number * 10 + int(character)
    continue
if character == ' ':
    continue
if operator == '+':
    stack.append(number)
elif operator == '-':
    stack.append(-number)
elif operator == '*':
    stack.append(stack.pop() * number)
else:
    stack.append(int(stack.pop() / number))
number = 0
operator = character
return sum(stack)
```

## Problem: 394. Decode String - Medium

```
class Solution:
    def decodeString(self, s):
        stack = []
        number = 0
        current = ''
        for character in s:
            if character.isdigit():
                number = number * 10 + int(character)
            elif character == '[':
                stack.append((current, number))
                current = ''
                number = 0
            elif character == ']':
                previous, repeat = stack.pop()
                current = previous + current * repeat
            else:
                current += character
        return current
```

## Problem: 71. Simplify Path - Medium

```
class Solution:
    def simplifyPath(self, path):
        stack = []
        for part in path.split('/'):
            if part in {'', '.'}:
                continue
            if part == '..':
                if stack:
                    stack.pop()
```

```
        else:
            stack.append(part)
    return '/' + '/'.join(stack)
```

## Problem: 402. Remove K Digits - Medium

```
class Solution:
    def removeKdigits(self, num, k):
        stack = []
        for digit in num:
            while k and stack and stack[-1] > digit:
                stack.pop()
                k -= 1
            stack.append(digit)
        if k:
            stack = stack[:-k]
        return ''.join(stack).lstrip('0') or '0'
```

## Problem: 316. Remove Duplicate Letters - Medium

```
class Solution:
    def removeDuplicateLetters(self, s):
        remaining = {character: s.count(character) for character in set(s)}
        stack = []
        used = set()
        for character in s:
            remaining[character] -= 1
            if character in used:
                continue
            while (stack and stack[-1] > character
                  and remaining[stack[-1]] > 0):
                used.remove(stack.pop())
            stack.append(character)
            used.add(character)
        return ''.join(stack)
```

## Problem: 1081. Smallest Subsequence of Distinct Characters - Medium

```
class Solution:
    def smallestSubsequence(self, s):
        remaining = {character: s.count(character) for character in set(s)}
        stack = []
        used = set()
        for character in s:
            remaining[character] -= 1
            if character in used:
```

```

        continue
    while stack and stack[-1] > character and remaining[stack[-1]]:
        used.remove(stack.pop())
    stack.append(character)
    used.add(character)
return ''.join(stack)

```

## Problem: 456. 132 Pattern - Medium

```

class Solution:
    def find132pattern(self, nums):
        stack = []
        middle = float('-inf')
        for value in reversed(nums):
            if value < middle:
                return True
            while stack and stack[-1] < value:
                middle = stack.pop()
            stack.append(value)
        return False

```

## Problem: 155. Min Stack - Medium

```

class MinStack:
    def __init__(self):
        self.stack = []
    def push(self, val):
        current_min = min(val, self.stack[-1][1]) if self.stack else val
        self.stack.append((val, current_min))
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1][0]
    def getMin(self):
        return self.stack[-1][1]

```

## Problem: 225. Implement Stack using Queues - Easy

```

from collections import deque
class MyStack:
    def __init__(self):
        self.queue = deque()
    def push(self, x):
        self.queue.append(x)
        for _ in range(len(self.queue) - 1):
            self.queue.append(self.queue.popleft())

```

```
def pop(self):
    return self.queue.popleft()
def top(self):
    return self.queue[0]
def empty(self):
    return not self.queue
```

## Problem: 232. Implement Queue using Stacks - Easy

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []
    def _move(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
    def push(self, x):
        self.in_stack.append(x)
    def pop(self):
        self._move()
        return self.out_stack.pop()
    def peek(self):
        self._move()
        return self.out_stack[-1]
    def empty(self):
        return not self.in_stack and not self.out_stack
```

## Problem: 388. Longest Absolute File Path - Medium

```
class Solution:
    def lengthLongestPath(self, input):
        lengths = {0: 0}
        best = 0
        for line in input.split('\n'):
            depth = line.count('\t')
            name = line.lstrip('\t')
            lengths[depth + 1] = lengths[depth] + len(name) + 1
            if '.' in name:
                best = max(best, lengths[depth + 1] - 1)
        return best
```

## Problem: 636. Exclusive Time of Functions - Medium

```
class Solution:
    def exclusiveTime(self, n, logs):
```

```
answer = [0] * n
stack = []
previous_time = 0
for log in logs:
    function_id, event, timestamp = log.split(':')
    function_id = int(function_id)
    timestamp = int(timestamp)
    if event == 'start':
        if stack:
            answer[stack[-1]] += timestamp - previous_time
            stack.append(function_id)
            previous_time = timestamp
        else:
            answer[stack.pop()] += timestamp - previous_time + 1
            previous_time = timestamp + 1
return answer
```

## Problem: 735. Asteroid Collision - Medium

```
class Solution:
    def asteroidCollision(self, asteroids):
        stack = []
        for asteroid in asteroids:
            alive = True
            while alive and asteroid < 0 and stack and stack[-1] > 0:
                if stack[-1] < -asteroid:
                    stack.pop()
                elif stack[-1] == -asteroid:
                    stack.pop()
                    alive = False
                else:
                    alive = False
            if alive:
                stack.append(asteroid)
        return stack
```

## Problem: 1. Two Sum

```
class Solution:
    def twoSum(self, nums, target):
        seen = {}
        for i, value in enumerate(nums):
            need = target - value
            if need in seen:
                return [seen[need], i]
            seen[value] = i
        return []
```

## Problem: 167. Two Sum II - Input Array Is Sorted

```
class Solution:
    def twoSum(self, numbers, target):
        left, right = 0, len(numbers) - 1
        while left < right:
            total = numbers[left] + numbers[right]
            if total == target:
                return [left + 1, right + 1]
            if total < target:
                left += 1
            else:
                right -= 1
        return []
```

## Problem: 15. 3Sum

```
class Solution:
    def threeSum(self, nums):
        nums.sort()
        result = []
        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            left, right = i + 1, len(nums) - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total == 0:
                    result.append([nums[i], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
                elif total < 0:
                    left += 1
                else:
                    right -= 1
        return result
```

## Problem: 16. 3Sum Closest

```
class Solution:
    def threeSumClosest(self, nums, target):
        nums.sort()
        best = float('inf')
```

```
for i in range(len(nums) - 2):
    left, right = i + 1, len(nums) - 1
    while left < right:
        total = nums[i] + nums[left] + nums[right]
        if abs(total - target) < abs(best - target):
            best = total
        if total < target:
            left += 1
        else:
            right -= 1
    return best
```

## Problem: 18. 4Sum

```
class Solution:
    def fourSum(self, nums, target):
        nums.sort()
        n = len(nums)
        result = []
        for i in range(n - 3):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            for j in range(i + 1, n - 2):
                if j > i + 1 and nums[j] == nums[j - 1]:
                    continue
                left, right = j + 1, n - 1
                while left < right:
                    total = nums[i] + nums[j] + nums[left] + nums[right]
                    if total == target:
                        result.append([nums[i], nums[j], nums[left], nums[right]])
                        left += 1
                        right -= 1
                        while left < right and nums[left] == nums[left - 1]:
                            left += 1
                        while left < right and nums[right] == nums[right + 1]:
                            right -= 1
                    elif total < target:
                        left += 1
                    else:
                        right -= 1
            return result
```

## Problem: 259. 3Sum Smaller

```
class Solution:
    def threeSumSmaller(self, nums, target):
        nums.sort()
        count = 0
        for i in range(len(nums) - 2):
```

```
        left, right = i + 1, len(nums) - 1
    while left < right:
        total = nums[i] + nums[left] + nums[right]
        if total < target:
            count += right - left
            left += 1
        else:
            right -= 1
    return count
```

## Problem: 611. Valid Triangle Number

```
class Solution:
    def triangleNumber(self, nums):
        nums.sort()
        count = 0
        for i in range(len(nums) - 2):
            left, right = i + 1, len(nums) - 1
            while left < right:
                if nums[i] + nums[left] > nums[right]:
                    count += right - left
                    right -= 1
                else:
                    left += 1
        return count
```

## Problem: 219. Contains Duplicate II

```
class Solution:
    def containsNearbyDuplicate(self, nums, k):
        seen = {}
        for i, value in enumerate(nums):
            if value in seen and i - seen[value] <= k:
                return True
            seen[value] = i
        return False
```

## Problem: 26. Remove Duplicates from Sorted Array

```
class Solution:
    def removeDuplicates(self, nums):
        if not nums:
            return 0
        write = 1
        for read in range(1, len(nums)):
            if nums[read] != nums[write - 1]:
                nums[write] = nums[read]
                write += 1
```

```
        nums[write] = nums[read]
        write += 1
    return write
```

## Problem: 27. Remove Element

```
class Solution:
    def removeElement(self, nums, val):
        write = 0
        for value in nums:
            if value != val:
                nums[write] = value
                write += 1
        return write
```

## Problem: 80. Remove Duplicates from Sorted Array II

```
class Solution:
    def removeDuplicates(self, nums):
        if len(nums) <= 2:
            return len(nums)
        write = 2
        for i in range(2, len(nums)):
            if nums[i] != nums[write - 2]:
                nums[write] = nums[i]
                write += 1
        return write
```

## Problem: 88. Merge Sorted Array

```
class Solution:
    def merge(self, nums1, m, nums2, n):
        i = m - 1
        j = n - 1
        k = m + n - 1
        while i >= 0 and j >= 0:
            if nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1
        while j >= 0:
            nums1[k] = nums2[j]
```

```
j -= 1
k -= 1
```

## Problem: 75. Sort Colors

```
class Solution:
    def sortColors(self, nums):
        low, mid, high = 0, 0, len(nums) - 1
        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low]
                low += 1
                mid += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[mid], nums[high] = nums[high], nums[mid]
                high -= 1
```

## Problem: 283. Move Zeroes

```
class Solution:
    def moveZeroes(self, nums):
        write = 0
        for value in nums:
            if value != 0:
                nums[write] = value
                write += 1
        for i in range(write, len(nums)):
            nums[i] = 0
```

## Problem: 977. Squares of a Sorted Array

```
class Solution:
    def sortedSquares(self, nums):
        result = [0] * len(nums)
        left, right = 0, len(nums) - 1
        index = len(nums) - 1
        while left <= right:
            left_square = nums[left] * nums[left]
            right_square = nums[right] * nums[right]
            if left_square > right_square:
                result[index] = left_square
                left += 1
            else:
                result[index] = right_square
```

```
        right -= 1
    index -= 1
    return result
```

## Problem: 344. Reverse String

```
class Solution:
    def reverseString(self, s):
        left, right = 0, len(s) - 1
        while left < right:
            s[left], s[right] = s[right], s[left]
            left += 1
            right -= 1
```

## Problem: 345. Reverse Vowels of a String

```
class Solution:
    def reverseVowels(self, s):
        vowels = set('aeiouAEIOU')
        chars = list(s)
        left, right = 0, len(chars) - 1
        while left < right:
            while left < right and chars[left] not in vowels:
                left += 1
            while left < right and chars[right] not in vowels:
                right -= 1
            if left < right:
                chars[left], chars[right] = chars[right], chars[left]
                left += 1
                right -= 1
        return ''.join(chars)
```

## Problem: 349. Intersection of Two Arrays

```
class Solution:
    def intersection(self, nums1, nums2):
        nums1 = sorted(set(nums1))
        nums2 = sorted(set(nums2))
        i = j = 0
        result = []
        while i < len(nums1) and j < len(nums2):
            if nums1[i] == nums2[j]:
                result.append(nums1[i])
                i += 1
                j += 1
            elif nums1[i] < nums2[j]:
```

```
        i += 1
    else:
        j += 1
    return result
```

## Problem: 350. Intersection of Two Arrays II

```
class Solution:
    def intersect(self, nums1, nums2):
        nums1.sort()
        nums2.sort()
        i = j = 0
        result = []
        while i < len(nums1) and j < len(nums2):
            if nums1[i] == nums2[j]:
                result.append(nums1[i])
                i += 1
                j += 1
            elif nums1[i] < nums2[j]:
                i += 1
            else:
                j += 1
        return result
```

## Problem: 141. Linked List Cycle

```
class Solution:
    def hasCycle(self, head):
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False
```

## Problem: 142. Linked List Cycle II

```
class Solution:
    def detectCycle(self, head):
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
```

```
        if slow == fast:
            break
    else:
        return None
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next
    return slow
```

## Problem: 876. Middle of the Linked List

```
class Solution:
    def middleNode(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

## Problem: 19. Remove Nth Node From End of List

```
class Solution:
    def removeNthFromEnd(self, head, n):
        dummy = ListNode(0)
        dummy.next = head
        fast = dummy
        slow = dummy
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next
```

## Problem: 234. Palindrome Linked List

```
class Solution:
    def isPalindrome(self, head):
        if not head or not head.next:
            return True
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
```

```
        fast = fast.next.next
    if fast:
        slow = slow.next
    prev = None
    while slow:
        nxt = slow.next
        slow.next = prev
        prev = slow
        slow = nxt
    while prev:
        if head.val != prev.val:
            return False
        head = head.next
        prev = prev.next
    return True
```

## Problem: 287. Find the Duplicate Number

```
class Solution:
    def findDuplicate(self, nums):
        slow = nums[0]
        fast = nums[0]
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow
```

## Problem: 202. Happy Number

```
class Solution:
    def isHappy(self, n):
        def next_number(x):
            total = 0
            while x:
                x, mod = divmod(x, 10)
                total += mod * mod
            return total
        slow = n
        fast = n
        while True:
            slow = next_number(slow)
            fast = next_number(next_number(fast))
            if slow == fast:
```

```
        break
    return slow == 1
```

## Problem: 204. Count Primes

```
import math
class Solution:
    def countPrimes(self, n):
        if n < 2:
            return 0
        is_prime = [True] * n
        is_prime[0] = is_prime[1] = False
        for p in range(2, int(math.sqrt(n)) + 1):
            if is_prime[p]:
                for multiple in range(p * p, n, p):
                    is_prime[multiple] = False
        return sum(is_prime)
```

## Problem: 125. Valid Palindrome

```
class Solution:
    def isPalindrome(self, s):
        left, right = 0, len(s) - 1
        while left < right:
            while left < right and not s[left].isalnum():
                left += 1
            while left < right and not s[right].isalnum():
                right -= 1
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

## Problem: 680. Valid Palindrome II

```
class Solution:
    def validPalindrome(self, s):
        def is_palindrome(lo, hi):
            while lo < hi:
                if s[lo] != s[hi]:
                    return False
                lo += 1
                hi -= 1
            return True
        left, right = 0, len(s) - 1
```

```
while left < right:
    if s[left] == s[right]:
        left += 1
        right -= 1
    else:
        return is_palindrome(left + 1, right) or is_palindrome(left, right - 1)
return True
```

## Problem: 7. Reverse Integer

```
class Solution:
    def reverse(self, x):
        sign = -1 if x < 0 else 1
        x = abs(x)
        result = 0
        while x:
            if result > (2**31 - 1) // 10:
                return 0
            result = result * 10 + x % 10
            x //= 10
        return sign * result
```

## Problem: 121. Best Time to Buy and Sell Stock

```
class Solution:
    def maxProfit(self, prices):
        if not prices:
            return 0
        min_price = prices[0]
        best = 0
        for price in prices[1:]:
            min_price = min(min_price, price)
            best = max(best, price - min_price)
        return best
```

## Problem: 122. Best Time to Buy and Sell Stock II

```
class Solution:
    def maxProfit(self, prices):
        profit = 0
        for i in range(1, len(prices)):
            if prices[i] > prices[i - 1]:
                profit += prices[i] - prices[i - 1]
        return profit
```

## Problem: 11. Container With Most Water

```
class Solution:
    def maxArea(self, height):
        left, right = 0, len(height) - 1
        best = 0
        while left < right:
            width = right - left
            best = max(best, min(height[left], height[right]) * width)
            if height[left] <= height[right]:
                left += 1
            else:
                right -= 1
        return best
```

## Problem: 21. Merge Two Sorted Lists

```
class Solution:
    def mergeTwoLists(self, l1, l2):
        dummy = ListNode(0)
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 if l1 else l2
        return dummy.next
```

## Problem: 23. Merge k Sorted Lists

```
import heapq
class Solution:
    def mergeKLists(self, lists):
        heap = []
        for i, head in enumerate(lists):
            if head:
                heapq.heappush(heap, (head.val, i, head))
        dummy = ListNode(0)
        tail = dummy
        while heap:
            val, index, node = heapq.heappop(heap)
            tail.next = node
            tail = tail.next
```

```

        if node.next:
            heapq.heappush(heap, (node.next.val, index, node.next))
    return dummy.next

```

## Problem: 315. Count of Smaller Numbers After Self

```

class Solution:
    def countSmaller(self, nums):
        n = len(nums)
        indexes = list(range(n))
        counts = [0] * n
        def merge_sort(left, right):
            if left >= right:
                return
            mid = (left + right) // 2
            merge_sort(left, mid)
            merge_sort(mid + 1, right)
            temp = []
            i, j = left, mid + 1
            while i <= mid and j <= right:
                if nums[indexes[i]] <= nums[indexes[j]]:
                    temp.append(indexes[i])
                    i += 1
                else:
                    counts[indexes[j]] += mid - i + 1
                    temp.append(indexes[j])
                    j += 1
            while i <= mid:
                temp.append(indexes[i])
                i += 1
            while j <= right:
                temp.append(indexes[j])
                j += 1
            indexes[left:right + 1] = temp
        merge_sort(0, n - 1)
        return counts

```

## Problem: 328. Odd Even Linked List

```

class Solution:
    def oddEvenList(self, head):
        if not head or not head.next:
            return head
        odd = head
        even = head.next
        even_head = even
        while even and even.next:
            odd.next = even.next
            odd = odd.next
            even = even.next

```

```
        even.next = odd.next
        even = even.next
    odd.next = even_head
    return head
```

## Problem: 206. Reverse Linked List

```
class Solution:
    def reverseList(self, head):
        prev = None
        curr = head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        return prev
```

## Problem: 92. Reverse Linked List II

```
class Solution:
    def reverseBetween(self, head, left, right):
        dummy = ListNode(0)
        dummy.next = head
        prev = dummy
        for _ in range(left - 1):
            prev = prev.next
        curr = prev.next
        for _ in range(right - left):
            nxt = curr.next
            curr.next = nxt.next
            nxt.next = prev.next
            prev.next = nxt
        return dummy.next
```

## Problem: 209. Minimum Size Subarray Sum

```
class Solution:
    def minSubArrayLen(self, target, nums):
        left = 0
        current = 0
        best = float('inf')
        for right, value in enumerate(nums):
            current += value
            while current >= target:
                best = min(best, right - left + 1)
                current -= nums[left]
                left += 1
```

```
        current -= nums[left]
        left += 1
    return 0 if best == float('inf') else best
```

## Problem: 713. Subarray Product Less Than K

```
class Solution:
    def numSubarrayProductLessThanK(self, nums, k):
        if k <= 1:
            return 0
        left = 0
        product = 1
        answer = 0
        for right, value in enumerate(nums):
            product *= value
            while product >= k:
                product /= nums[left]
                left += 1
            answer += right - left + 1
        return answer
```

## Problem: 438. Find All Anagrams in a String

```
from collections import Counter
class Solution:
    def findAnagrams(self, s, p):
        need = Counter(p)
        window = Counter()
        left = 0
        result = []
        for right, ch in enumerate(s):
            window[ch] += 1
            while right - left + 1 > len(p):
                left_ch = s[left]
                window[left_ch] -= 1
                if window[left_ch] == 0:
                    del window[left_ch]
                left += 1
            if window == need:
                result.append(left)
        return result
```

## Problem: 3. Longest Substring Without Repeating Characters

```
class Solution:
    def lengthOfLongestSubstring(self, s):
```

```

left = 0
seen = {}
best = 0
for right, ch in enumerate(s):
    if ch in seen and seen[ch] >= left:
        left = seen[ch] + 1
    seen[ch] = right
    best = max(best, right - left + 1)
return best

```

## Problem: 395. Longest Substring with At Least K Repeating Characters

```

from collections import Counter
class Solution:
    def longestSubstring(self, s, k):
        if len(s) < k:
            return 0
        counts = Counter(s)
        for i, ch in enumerate(s):
            if counts[ch] < k:
                return max(self.longestSubstring(s[:i], k),
self.longestSubstring(s[i + 1:], k))
        return len(s)

```

## Problem: 1004. Max Consecutive Ones III

```

class Solution:
    def longestOnes(self, nums, k):
        left = 0
        zeros = 0
        best = 0
        for right, value in enumerate(nums):
            if value == 0:
                zeros += 1
            while zeros > k:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            best = max(best, right - left + 1)
        return best

```

## Problem: 42. Trapping Rain Water

```

class Solution:
    def trap(self, height):
        left, right = 0, len(height) - 1

```

```
left_max = 0
right_max = 0
water = 0
while left < right:
    if height[left] <= height[right]:
        if height[left] >= left_max:
            left_max = height[left]
        else:
            water += left_max - height[left]
        left += 1
    else:
        if height[right] >= right_max:
            right_max = height[right]
        else:
            water += right_max - height[right]
        right -= 1
return water
```

## Problem: 763. Partition Labels

```
class Solution:
    def partitionLabels(self, s):
        last = {ch: i for i, ch in enumerate(s)}
        result = []
        start = 0
        end = 0
        for i, ch in enumerate(s):
            end = max(end, last[ch])
            if i == end:
                result.append(end - start + 1)
                start = i + 1
        return result
```

## Problem: 1329. Sort the Matrix Diagonally

```
from collections import defaultdict
class Solution:
    def diagonalSort(self, mat):
        groups = defaultdict(list)
        for i in range(len(mat)):
            for j in range(len(mat[0])):
                groups[i - j].append(mat[i][j])
        for key in groups:
            groups[key].sort()
        for i in range(len(mat)):
            for j in range(len(mat[0])):
                mat[i][j] = groups[i - j].pop(0)
        return mat
```

## Problem: 986. Interval List Intersections

```
class Solution:
    def intervalIntersection(self, firstList, secondList):
        i = j = 0
        result = []
        while i < len(firstList) and j < len(secondList):
            lo = max(firstList[i][0], secondList[j][0])
            hi = min(firstList[i][1], secondList[j][1])
            if lo <= hi:
                result.append([lo, hi])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1
        return result
```

## Problem: 228. Summary Ranges

```
class Solution:
    def summaryRanges(self, nums):
        if not nums:
            return []
        result = []
        start = prev = nums[0]
        for value in nums[1:]:
            if value == prev + 1:
                prev = value
            else:
                if start == prev:
                    result.append(str(start))
                else:
                    result.append(f"{start}->{prev}")
```

```
        start = prev = value
    if start == prev:
        result.append(str(start))
    else:
        result.append(f"{start}->{prev}")
    return result
```

## Problem: 53. Maximum Subarray

```
class Solution:
    def maxSubArray(self, nums):
        best = nums[0]
        current = nums[0]
        for value in nums[1:]:
            current = max(value, current + value)
            best = max(best, current)
        return best
```

## Problem: 152. Maximum Product Subarray

```
class Solution:
    def maxProduct(self, nums):
        best = nums[0]
        curr_max = nums[0]
        curr_min = nums[0]
        for value in nums[1:]:
            prev_max, prev_min = curr_max, curr_min
            curr_max = max(value, prev_max * value, prev_min * value)
            curr_min = min(value, prev_max * value, prev_min * value)
            best = max(best, curr_max)
        return best
```

## Problem: 31. Next Permutation

```
class Solution:
    def nextPermutation(self, nums):
        n = len(nums)
        i = n - 2
        while i >= 0 and nums[i] >= nums[i + 1]:
            i -= 1
        if i >= 0:
            j = n - 1
            while nums[j] <= nums[i]:
                j -= 1
            nums[i], nums[j] = nums[j], nums[i]
        left, right = i + 1, n - 1
        while left < right:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

```

while left < right:
    nums[left], nums[right] = nums[right], nums[left]
    left += 1
    right -= 1

```

## Problem: 5. Longest Palindromic Substring - Medium

```

class Solution:
    def longestPalindrome(self, s):
        best_start = 0
        best_length = 1 if s else 0
        def expand(left, right):
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1
            return left + 1, right - left - 1
        for center in range(len(s)):
            odd_start, odd_length = expand(center, center)
            even_start, even_length = expand(center, center + 1)
            if odd_length > best_length:
                best_start, best_length = odd_start, odd_length
            if even_length > best_length:
                best_start, best_length = even_start, even_length
        return s[best_start:best_start + best_length]

```

## Problem: 28. Find the Index of the First Occurrence in a String - Easy

```

class Solution:
    def strStr(self, haystack, needle):
        if not needle:
            return 0
        for start in range(len(haystack) - len(needle) + 1):
            if haystack[start:start + len(needle)] == needle:
                return start
        return -1

```

## Problem: 61. Rotate List - Medium

```

class Solution:
    def rotateRight(self, head, k):
        if not head or not head.next or k == 0:
            return head
        length = 1
        tail = head
        while tail.next:
            tail = tail.next

```

```

        length += 1
    k %= length
    if k == 0:
        return head
    tail.next = head
    steps_to_new_tail = length - k
    new_tail = tail
    for _ in range(steps_to_new_tail):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    return new_head

```

## Problem: 82. Remove Duplicates from Sorted List II - Medium

```

class Solution:
    def deleteDuplicates(self, head):
        dummy = ListNode(0)
        dummy.next = head
        prev = dummy
        while head:
            if head.next and head.val == head.next.val:
                duplicate = head.val
                while head and head.val == duplicate:
                    head = head.next
                prev.next = head
            else:
                prev = head
                head = head.next
        return dummy.next

```

## Problem: 86. Partition List - Medium

```

class Solution:
    def partition(self, head, x):
        before = ListNode(0)
        after = ListNode(0)
        before_tail = before
        after_tail = after
        while head:
            if head.val < x:
                before_tail.next = head
                before_tail = before_tail.next
            else:
                after_tail.next = head
                after_tail = after_tail.next
            head = head.next
        after_tail.next = None

```

```
before_tail.next = after.next
return before.next
```

## Problem: 143. Reorder List - Medium

```
class Solution:
    def reorderList(self, head):
        if not head or not head.next:
            return
        slow = fast = head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next
        second = slow.next
        slow.next = None
        previous = None
        while second:
            next_node = second.next
            second.next = previous
            previous = second
            second = next_node
        first = head
        second = previous
        while second:
            first_next = first.next
            second_next = second.next
            first.next = second
            second.next = first_next
            first = first_next
            second = second_next
```

## Problem: 148. Sort List - Medium

```
class Solution:
    def sortList(self, head):
        if not head or not head.next:
            return head
        slow = head
        fast = head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        second = slow.next
        slow.next = None
        left = self.sortList(head)
        right = self.sortList(second)
        return self.merge(left, right)
    def merge(self, left, right):
        dummy = ListNode(0)
```

```
tail = dummy
while left and right:
    if left.val <= right.val:
        tail.next = left
        left = left.next
    else:
        tail.next = right
        right = right.next
    tail = tail.next
tail.next = left if left else right
return dummy.next
```

## Problem: 151. Reverse Words in a String - Medium

```
class Solution:
    def reverseWords(self, s):
        words = s.split()
        left, right = 0, len(words) - 1
        while left < right:
            words[left], words[right] = words[right], words[left]
            left += 1
            right -= 1
        return ' '.join(words)
```

## Problem: 160. Intersection of Two Linked Lists - Easy

```
class Solution:
    def getIntersectionNode(self, headA, headB):
        pointer_a = headA
        pointer_b = headB
        while pointer_a != pointer_b:
            pointer_a = pointer_a.next if pointer_a else headB
            pointer_b = pointer_b.next if pointer_b else headA
        return pointer_a
```

## Problem: 161. One Edit Distance - Medium

```
class Solution:
    def isOneEditDistance(self, s, t):
        if abs(len(s) - len(t)) > 1:
            return False
        if len(s) > len(t):
            s, t = t, s
        for i in range(len(s)):
            if s[i] != t[i]:
                if len(s) == len(t):
```

```
        return s[i + 1:] == t[i + 1:]
    return s[i:] == t[i + 1:]
return len(t) == len(s) + 1
```

## Problem: 165. Compare Version Numbers - Medium

```
class Solution:
    def compareVersion(self, version1, version2):
        parts1 = version1.split('.')
        parts2 = version2.split('.')
        length = max(len(parts1), len(parts2))
        for i in range(length):
            value1 = int(parts1[i]) if i < len(parts1) else 0
            value2 = int(parts2[i]) if i < len(parts2) else 0
            if value1 < value2:
                return -1
            if value1 > value2:
                return 1
        return 0
```

## Problem: 170. Two Sum III - Data structure design - Easy

```
class TwoSum:
    def __init__(self):
        self.counts = {}
    def add(self, number):
        self.counts[number] = self.counts.get(number, 0) + 1
    def find(self, value):
        for number, count in self.counts.items():
            complement = value - number
            if complement != number and complement in self.counts:
                return True
            if complement == number and count > 1:
                return True
        return False
```

## Problem: 186. Reverse Words in a String II - Medium

```
class Solution:
    def reverseWords(self, s):
        def reverse(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1
                right -= 1
        reverse(0, len(s) - 1)
```

```

start = 0
for end in range(len(s) + 1):
    if end == len(s) or s[end] == ' ':
        reverse(start, end - 1)
        start = end + 1

```

## Problem: 189. Rotate Array - Medium

```

class Solution:
    def rotate(self, nums, k):
        n = len(nums)
        k %= n
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1
                right -= 1
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)

```

## Problem: 244. Shortest Word Distance II - Medium

```

class WordDistance:
    def __init__(self, wordsDict):
        self.positions = {}
        for index, word in enumerate(wordsDict):
            self.positions.setdefault(word, []).append(index)
    def shortest(self, word1, word2):
        first = self.positions[word1]
        second = self.positions[word2]
        i = j = 0
        best = float('inf')
        while i < len(first) and j < len(second):
            best = min(best, abs(first[i] - second[j]))
            if first[i] < second[j]:
                i += 1
            else:
                j += 1
        return best

```

## Problem: 246. Strobogrammatic Number - Easy

```

class Solution:
    def isStrobogrammatic(self, num):
        pairs = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}

```

```
left, right = 0, len(num) - 1
while left <= right:
    if num[left] not in pairs or pairs[num[left]] != num[right]:
        return False
    left += 1
    right -= 1
return True
```

## Problem: 251. Flatten 2D Vector - Medium

```
class Vector2D:
    def __init__(self, vec):
        self.vec = vec
        self.row = 0
        self.column = 0
        self._skip_empty_rows()
    def _skip_empty_rows(self):
        while self.row < len(self.vec) and self.column >= len(self.vec[self.row]):
            self.row += 1
            self.column = 0
    def next(self):
        value = self.vec[self.row][self.column]
        self.column += 1
        self._skip_empty_rows()
        return value
    def hasNext(self):
        self._skip_empty_rows()
        return self.row < len(self.vec)
```

## Problem: 253. Meeting Rooms II - Medium

```
class Solution:
    def minMeetingRooms(self, intervals):
        starts = sorted(interval[0] for interval in intervals)
        ends = sorted(interval[1] for interval in intervals)
        start = end = 0
        rooms = 0
        while start < len(starts):
            if starts[start] < ends[end]:
                rooms += 1
                start += 1
            else:
                end += 1
                start += 1
        return rooms
```

## Problem: 272. Closest Binary Search Tree Value II - Hard

```

from collections import deque
class Solution:
    def closestKValues(self, root, target, k):
        values = []
        def inorder(node):
            if not node:
                return
            inorder(node.left)
            values.append(node.val)
            inorder(node.right)
        inorder(root)
        values.sort(key=lambda value: abs(value - target))
        return values[:k]

```

## Problem: 277. Find the Celebrity - Medium

```

class Solution:
    def findCelebrity(self, n):
        candidate = 0
        for person in range(1, n):
            if knows(candidate, person):
                candidate = person
        for person in range(n):
            if person == candidate:
                continue
            if knows(candidate, person) or not knows(person, candidate):
                return -1
        return candidate

```

## Problem: 295. Find Median from Data Stream - Hard

```

import heapq
class MedianFinder:
    def __init__(self):
        self.lower = []
        self.upper = []
    def addNum(self, num):
        heapq.heappush(self.lower, -num)
        heapq.heappush(self.upper, -heapq.heappop(self.lower))
        if len(self.upper) > len(self.lower):
            heapq.heappush(self.lower, -heapq.heappop(self.upper))
    def findMedian(self):
        if len(self.lower) > len(self.upper):
            return -self.lower[0]
        return (-self.lower[0] + self.upper[0]) / 2

```

## Problem: 321. Create Maximum Number - Hard

```
class Solution:
    def maxNumber(self, nums1, nums2, k):
        best = []
        for take in range(max(0, k - len(nums2)), min(k, len(nums1)) + 1):
            first = self.pick(nums1, take)
            second = self.pick(nums2, k - take)
            candidate = self.merge(first, second)
            best = max(best, candidate)
        return best

    def pick(self, nums, length):
        drop = len(nums) - length
        stack = []
        for value in nums:
            while drop and stack and stack[-1] < value:
                stack.pop()
                drop -= 1
            stack.append(value)
        return stack[:length]

    def merge(self, first, second):
        result = []
        while first or second:
            if first > second:
                result.append(first.pop(0))
            else:
                result.append(second.pop(0))
        return result
```

## Problem: 360. Sort Transformed Array - Medium

```
class Solution:
    def sortTransformedArray(self, nums, a, b, c):
        def transform(value):
            return a * value * value + b * value + c
        result = [0] * len(nums)
        left, right = 0, len(nums) - 1
        if a >= 0:
            index = len(nums) - 1
            while left <= right:
                left_value = transform(nums[left])
                right_value = transform(nums[right])
                if left_value > right_value:
                    result[index] = left_value
                    left += 1
                else:
                    result[index] = right_value
                    right -= 1
                index -= 1
        else:
```

```
index = 0
while left <= right:
    left_value = transform(nums[left])
    right_value = transform(nums[right])
    if left_value < right_value:
        result[index] = left_value
        left += 1
    else:
        result[index] = right_value
        right -= 1
    index += 1
return result
```

## Problem: 392. Is Subsequence - Easy

```
class Solution:
    def isSubsequence(self, s, t):
        i = 0
        for character in t:
            if i < len(s) and s[i] == character:
                i += 1
        return i == len(s)
```

## Problem: 408. Valid Word Abbreviation - Easy

```
class Solution:
    def validWordAbbreviation(self, word, abbr):
        word_index = 0
        abbr_index = 0
        while word_index < len(word) and abbr_index < len(abbr):
            if abbr[abbr_index].isalpha():
                if word[word_index] != abbr[abbr_index]:
                    return False
                word_index += 1
                abbr_index += 1
                continue
            if abbr[abbr_index] == '0':
                return False
            number = 0
            while abbr_index < len(abbr) and abbr[abbr_index].isdigit():
                number = number * 10 + int(abbr[abbr_index])
                abbr_index += 1
            word_index += number
        return word_index == len(word) and abbr_index == len(abbr)
```

## Problem: 443. String Compression - Medium

```
class Solution:
    def compress(self, chars):
        write = 0
        read = 0
        while read < len(chars):
            character = chars[read]
            start = read
            while read < len(chars) and chars[read] == character:
                read += 1
            chars[write] = character
            write += 1
            count = read - start
            if count > 1:
                for digit in str(count):
                    chars[write] = digit
                    write += 1
        return write
```

## Problem: 455. Assign Cookies - Easy

```
class Solution:
    def findContentChildren(self, greed, cookies):
        greed.sort()
        cookies.sort()
        child = cookie = 0
        while child < len(greed) and cookie < len(cookies):
            if cookies[cookie] >= greed[child]:
                child += 1
            cookie += 1
        return child
```

## Problem: 457. Circular Array Loop - Medium

```
class Solution:
    def circularArrayLoop(self, nums):
        n = len(nums)
        def next_index(index):
            return (index + nums[index]) % n
        def same_direction(index, direction):
            return nums[index] * direction > 0
        for start in range(n):
            if nums[start] == 0:
                continue
            direction = nums[start]
            slow = start
            fast = start
            while (same_direction(slow, direction)
```

```

        and same_direction(next_index(fast), direction)):
    slow = next_index(slow)
    fast = next_index(next_index(fast))
    if slow == fast:
        if slow == next_index(slow):
            break
        return True
    index = start
    while same_direction(index, direction):
        next_node = next_index(index)
        nums[index] = 0
        index = next_node
    return False

```

## Problem: 466. Count The Repetitions - Hard

```

class Solution:
    def getMaxRepetitions(self, s1, n1, s2, n2):
        if n1 == 0:
            return 0
        index = 0
        count1 = 0
        count2 = 0
        seen = {}
        gained = {}
        while count1 < n1:
            for character in s1:
                if character == s2[index]:
                    index += 1
                    if index == len(s2):
                        index = 0
                        count2 += 1
            count1 += 1
            if index in seen:
                previous_count1 = seen[index]
                previous_count2 = gained[index]
                cycle1 = count1 - previous_count1
                cycle2 = count2 - previous_count2
                remaining = n1 - count1
                cycles = remaining // cycle1
                count1 += cycles * cycle1
                count2 += cycles * cycle2
            else:
                seen[index] = count1
                gained[index] = count2
        return count2 // n2

```

## Problem: 475. Heaters - Medium

```
class Solution:
    def findRadius(self, houses, heaters):
        houses.sort()
        heaters.sort()
        heater = 0
        radius = 0
        for house in houses:
            while (heater + 1 < len(heaters)
                  and abs(heaters[heater + 1] - house) <= abs(heaters[heater] -
house)):
                heater += 1
            radius = max(radius, abs(heaters[heater] - house))
        return radius
```

## Problem: 481. Magical String - Medium

```
class Solution:
    def magicalString(self, n):
        if n <= 0:
            return 0
        if n <= 3:
            return 1
        sequence = [1, 2, 2]
        read = 2
        next_value = 1
        while len(sequence) < n:
            sequence.extend([next_value] * sequence[read])
            next_value = 3 - next_value
            read += 1
        return sequence[:n].count(1)
```

## Problem: 522. Longest Uncommon Subsequence II - Medium

```
class Solution:
    def findLUSlength(self, strs):
        def is_subsequence(shorter, longer):
            i = 0
            for character in longer:
                if i < len(shorter) and shorter[i] == character:
                    i += 1
            return i == len(shorter)
        best = -1
        for i, candidate in enumerate(strs):
            if len(candidate) <= best:
                continue
            if all(i == j or not is_subsequence(candidate, other)
                  for j, other in enumerate(strs)):
```

```
        best = len(candidate)
    return best
```

## Problem: 524. Longest Word in Dictionary through Deleting - Medium

```
class Solution:
    def findLongestWord(self, s, dictionary):
        def is_subsequence(word):
            i = 0
            for character in s:
                if i < len(word) and word[i] == character:
                    i += 1
            return i == len(word)
        best = ''
        for word in dictionary:
            if is_subsequence(word) and (len(word) > len(best)
                                         or len(word) == len(best) and word <
best):
                best = word
        return best
```

## Problem: 532. K-diff Pairs in an Array - Medium

```
class Solution:
    def findPairs(self, nums, k):
        if k < 0:
            return 0
        nums.sort()
        left, right = 0, 1
        pairs = 0
        while right < len(nums):
            if left == right or nums[right] - nums[left] < k:
                right += 1
            elif nums[right] - nums[left] > k:
                left += 1
            else:
                pairs += 1
                left += 1
                right += 1
                while right < len(nums) and nums[right] == nums[right - 1]:
                    right += 1
        return pairs
```

## Problem: 541. Reverse String II - Easy

```
class Solution:
    def reverseStr(self, s, k):
        chars = list(s)
        for start in range(0, len(chars), 2 * k):
            left = start
            right = min(start + k - 1, len(chars) - 1)
            while left < right:
                chars[left], chars[right] = chars[right], chars[left]
                left += 1
                right -= 1
        return ''.join(chars)
```

## Problem: 556. Next Greater Element III - Medium

```
class Solution:
    def nextGreaterElement(self, n):
        digits = list(str(n))
        i = len(digits) - 2
        while i >= 0 and digits[i] >= digits[i + 1]:
            i -= 1
        if i < 0:
            return -1
        j = len(digits) - 1
        while digits[j] <= digits[i]:
            j -= 1
        digits[i], digits[j] = digits[j], digits[i]
        left, right = i + 1, len(digits) - 1
        while left < right:
            digits[left], digits[right] = digits[right], digits[left]
            left += 1
            right -= 1
        result = int(''.join(digits))
        return result if result < 2**31 else -1
```

## Problem: 557. Reverse Words in a String III - Easy

```
class Solution:
    def reverseWords(self, s):
        chars = list(s)
        start = 0
        for end in range(len(chars) + 1):
            if end == len(chars) or chars[end] == ' ':
                left, right = start, end - 1
                while left < right:
                    chars[left], chars[right] = chars[right], chars[left]
                    left += 1
                    right -= 1
                start = end + 1
```

```
        start = end + 1
    return ''.join(chars)
```

## Problem: 567. Permutation in String - Medium

```
from collections import Counter
class Solution:
    def checkInclusion(self, s1, s2):
        if len(s1) > len(s2):
            return False
        need = Counter(s1)
        window = Counter(s2[:len(s1)])
        if window == need:
            return True
        for right in range(len(s1), len(s2)):
            window[s2[right]] += 1
            left_character = s2[right - len(s1)]
            window[left_character] -= 1
            if window[left_character] == 0:
                del window[left_character]
            if window == need:
                return True
        return False
```

## Problem: 581. Shortest Unsorted Continuous Subarray - Medium

```
class Solution:
    def findUnsortedSubarray(self, nums):
        left, right = 0, len(nums) - 1
        while left < right and nums[left] <= nums[left + 1]:
            left += 1
        if left == right:
            return 0
        while right > left and nums[right - 1] <= nums[right]:
            right -= 1
        middle_min = min(nums[left:right + 1])
        middle_max = max(nums[left:right + 1])
        while left > 0 and nums[left - 1] > middle_min:
            left -= 1
        while right + 1 < len(nums) and nums[right + 1] < middle_max:
            right += 1
        return right - left + 1
```

## Problem: 633. Sum of Square Numbers - Medium

```
import math
class Solution:
    def judgeSquareSum(self, c):
        left = 0
        right = math.isqrt(c)
        while left <= right:
            total = left * left + right * right
            if total == c:
                return True
            if total < c:
                left += 1
            else:
                right -= 1
        return False
```

## Problem: 647. Palindromic Substrings - Medium

```
class Solution:
    def countSubstrings(self, s):
        count = 0
        def expand(left, right):
            nonlocal count
            while left >= 0 and right < len(s) and s[left] == s[right]:
                count += 1
                left -= 1
                right += 1
        for center in range(len(s)):
            expand(center, center)
            expand(center, center + 1)
        return count
```

## Problem: 653. Two Sum IV - Input is a BST - Easy

```
class Solution:
    def findTarget(self, root, k):
        seen = set()
        def search(node):
            if not node:
                return False
            if k - node.val in seen:
                return True
            seen.add(node.val)
            return search(node.left) or search(node.right)
        return search(root)
```

## Problem: 658. Find K Closest Elements - Medium

```
class Solution:
    def findClosestElements(self, arr, k, x):
        left, right = 0, len(arr) - k
        while left < right:
            middle = (left + right) // 2
            if x - arr[middle] > arr[middle + k] - x:
                left = middle + 1
            else:
                right = middle
        return arr[left:left + k]
```

## Problem: 696. Count Binary Substrings - Easy

```
class Solution:
    def countBinarySubstrings(self, s):
        previous_group = 0
        current_group = 1
        answer = 0
        for i in range(1, len(s)):
            if s[i] == s[i - 1]:
                current_group += 1
            else:
                answer += min(previous_group, current_group)
                previous_group = current_group
                current_group = 1
        return answer + min(previous_group, current_group)
```

## Problem: 719. Find K-th Smallest Pair Distance - Hard

```
class Solution:
    def smallestDistancePair(self, nums, k):
        nums.sort()
        left, right = 0, nums[-1] - nums[0]
        def count_at_most(distance):
            count = 0
            start = 0
            for end in range(len(nums)):
                while nums[end] - nums[start] > distance:
                    start += 1
                count += end - start
            return count
        while left < right:
            middle = (left + right) // 2
            if count_at_most(middle) >= k:
                right = middle
            else:
                left = middle + 1
```

```

        left = middle + 1
    return left

```

## Problem: 723. Candy Crush - Medium

```

class Solution:
    def candyCrush(self, board):
        rows, columns = len(board), len(board[0])
        changed = True
        while changed:
            changed = False
            crush = [[False] * columns for _ in range(rows)]
            for row in range(rows):
                for column in range(columns - 2):
                    value = abs(board[row][column])
                    if value and value == abs(board[row][column + 1]) ==
abs(board[row][column + 2]):
                        crush[row][column] = crush[row][column + 1] = crush[row]
[column + 2] = True
            for row in range(rows - 2):
                for column in range(columns):
                    value = abs(board[row][column])
                    if value and value == abs(board[row + 1][column]) ==
abs(board[row + 2][column]):
                        crush[row][column] = crush[row + 1][column] = crush[row +
2][column] = True
            for row in range(rows):
                for column in range(columns):
                    if crush[row][column]:
                        board[row][column] = 0
                        changed = True
            for column in range(columns):
                write = rows - 1
                for row in range(rows - 1, -1, -1):
                    if board[row][column]:
                        board[write][column] = board[row][column]
                        write -= 1
                while write >= 0:
                    board[write][column] = 0
                    write -= 1
        return board

```

## Problem: 777. Swap Adjacent in LR String - Medium

```

class Solution:
    def canTransform(self, start, result):
        if start.replace('X', '') != result.replace('X', ''):
            return False
        start_positions = [(character, index) for index, character in

```

```

enumerate(start) if character != 'X']
    result_positions = [(character, index) for index, character in
enumerate(result) if character != 'X']
    for (start_character, start_index), (result_character, result_index) in
zip(start_positions, result_positions):
        if start_character != result_character:
            return False
        if start_character == 'L' and start_index < result_index:
            return False
        if start_character == 'R' and start_index > result_index:
            return False
    return True

```

## Problem: 786. K-th Smallest Prime Fraction - Medium

```

class Solution:
    def kthSmallestPrimeFraction(self, arr, k):
        left, right = 0.0, 1.0
        while True:
            middle = (left + right) / 2
            count = 0
            best_numerator = 0
            best_denominator = 1
            numerator_index = 0
            for denominator_index in range(1, len(arr)):
                while numerator_index < denominator_index and arr[numerator_index]
/ arr[denominator_index] < middle:
                    numerator_index += 1
                count += numerator_index
                if numerator_index and arr[numerator_index - 1] * best_denominator
> best_numerator * arr[denominator_index]:
                    best_numerator = arr[numerator_index - 1]
                    best_denominator = arr[denominator_index]
            if count == k:
                return [best_numerator, best_denominator]
            if count < k:
                left = middle
            else:
                right = middle

```

## Problem: 795. Number of Subarrays with Bounded Maximum - Medium

```

class Solution:
    def numSubarrayBoundedMax(self, nums, left, right):
        answer = 0
        valid_start = 0
        last_too_large = -1
        for index, value in enumerate(nums):
            if value > right:

```

```

        last_too_large = index
    if value >= left:
        valid_start = index
    answer += valid_start - last_too_large
return answer

```

## Problem: 809. Expressive Words - Medium

```

class Solution:
    def expressiveWords(self, s, words):
        def is_stretchy(word):
            i = j = 0
            while i < len(s) and j < len(word):
                if s[i] != word[j]:
                    return False
                start_i, start_j = i, j
                while i < len(s) and s[i] == s[start_i]:
                    i += 1
                while j < len(word) and word[j] == word[start_j]:
                    j += 1
                source_count = i - start_i
                word_count = j - start_j
                if source_count < word_count or source_count < 3 and source_count
                != word_count:
                    return False
            return i == len(s) and j == len(word)
        return sum(is_stretchy(word) for word in words)

```

## Problem: 821. Shortest Distance to a Character - Easy

```

class Solution:
    def shortestToChar(self, s, c):
        distances = [len(s)] * len(s)
        previous = -len(s)
        for i, character in enumerate(s):
            if character == c:
                previous = i
                distances[i] = i - previous
        previous = 2 * len(s)
        for i in range(len(s) - 1, -1, -1):
            if s[i] == c:
                previous = i
                distances[i] = min(distances[i], previous - i)
        return distances

```

## Problem: 825. Friends Of Appropriate Ages - Medium

```
class Solution:
    def numFriendRequests(self, ages):
        counts = [0] * 121
        for age in ages:
            counts[age] += 1
        answer = 0
        for sender in range(15, 121):
            if counts[sender] == 0:
                continue
            for receiver in range(15, 121):
                if counts[receiver] == 0:
                    continue
                if receiver <= 0.5 * sender + 7 or receiver > sender or receiver > 100 and sender < 100:
                    continue
                answer += counts[sender] * (counts[receiver] - (sender == receiver))
        return answer
```

## Problem: 826. Most Profit Assigning Work - Medium

```
class Solution:
    def maxProfitAssignment(self, difficulty, profit, worker):
        jobs = sorted(zip(difficulty, profit))
        worker.sort()
        best_profit = 0
        job_index = 0
        answer = 0
        for ability in worker:
            while job_index < len(jobs) and jobs[job_index][0] <= ability:
                best_profit = max(best_profit, jobs[job_index][1])
                job_index += 1
            answer += best_profit
        return answer
```

## Problem: 832. Flipping an Image - Easy

```
class Solution:
    def flipAndInvertImage(self, image):
        for row in image:
            left, right = 0, len(row) - 1
            while left <= right:
                row[left], row[right] = 1 - row[right], 1 - row[left]
                left += 1
                right -= 1
        return image
```

## Problem: 838. Push Dominoes - Medium

```
class Solution:
    def pushDominoes(self, dominoes):
        symbols = ['L'] + list(dominoes) + ['R']
        left = 0
        for right in range(1, len(symbols)):
            if symbols[right] == '.':
                continue
            distance = right - left - 1
            if symbols[left] == symbols[right]:
                for i in range(left + 1, right):
                    symbols[i] = symbols[left]
            elif symbols[left] == 'R' and symbols[right] == 'L':
                for i in range(distance // 2 + 1):
                    symbols[left + 1 + i] = 'R'
                    symbols[right - 1 - i] = 'L'
            elif symbols[left] == 'L' and symbols[right] == 'R':
                pass
            left = right
        return ''.join(symbols[1:-1])
```

## Problem: 844. Backspace String Compare - Easy

```
class Solution:
    def backspaceCompare(self, s, t):
        def next_valid_index(text, index):
            skips = 0
            while index >= 0:
                if text[index] == '#':
                    skips += 1
                elif skips:
                    skips -= 1
                else:
                    return index
                index -= 1
            return -1
        i, j = len(s) - 1, len(t) - 1
        while True:
            i = next_valid_index(s, i)
            j = next_valid_index(t, j)
            if i < 0 or j < 0:
                return i == j
            if s[i] != t[j]:
                return False
            i -= 1
            j -= 1
```

## Problem: 845. Longest Mountain in Array - Medium

```
class Solution:
    def longestMountain(self, arr):
        best = 0
        i = 1
        while i < len(arr) - 1:
            if not (arr[i - 1] < arr[i] > arr[i + 1]):
                i += 1
                continue
            left = i - 1
            right = i + 1
            while left > 0 and arr[left - 1] < arr[left]:
                left -= 1
            while right + 1 < len(arr) and arr[right] > arr[right + 1]:
                right += 1
            best = max(best, right - left + 1)
            i = right
        return best
```

## Problem: 870. Advantage Shuffle - Medium

```
class Solution:
    def advantageCount(self, nums1, nums2):
        nums1.sort()
        order = sorted(range(len(nums2)), key=lambda i: nums2[i])
        result = [0] * len(nums2)
        left, right = 0, len(nums1) - 1
        for index in reversed(order):
            if nums1[right] > nums2[index]:
                result[index] = nums1[right]
                right -= 1
            else:
                result[index] = nums1[left]
                left += 1
        return result
```

## Problem: 881. Boats to Save People - Medium

```
class Solution:
    def numRescueBoats(self, people, limit):
        people.sort()
        left, right = 0, len(people) - 1
        boats = 0
        while left <= right:
            if people[left] + people[right] <= limit:
                left += 1
            right -= 1
            boats += 1
```

```
boats += 1
return boats
```

## Problem: 905. Sort Array By Parity - Easy

```
class Solution:
    def sortArrayByParity(self, nums):
        left, right = 0, len(nums) - 1
        while left < right:
            while left < right and nums[left] % 2 == 0:
                left += 1
            while left < right and nums[right] % 2 == 1:
                right -= 1
            if left < right:
                nums[left], nums[right] = nums[right], nums[left]
        return nums
```

## Problem: 917. Reverse Only Letters - Easy

```
class Solution:
    def reverseOnlyLetters(self, s):
        chars = list(s)
        left, right = 0, len(chars) - 1
        while left < right:
            while left < right and not chars[left].isalpha():
                left += 1
            while left < right and not chars[right].isalpha():
                right -= 1
            if left < right:
                chars[left], chars[right] = chars[right], chars[left]
                left += 1
                right -= 1
        return ''.join(chars)
```

## Problem: 922. Sort Array By Parity II - Easy

```
class Solution:
    def sortArrayByParityII(self, nums):
        even = 0
        odd = 1
        while even < len(nums) and odd < len(nums):
            if nums[even] % 2 == 0:
                even += 2
            elif nums[odd] % 2 == 1:
                odd += 2
            else:
```

```
        nums[even], nums[odd] = nums[odd], nums[even]
        even += 2
        odd += 2
    return nums
```

## Problem: 923. 3Sum With Multiplicity - Medium

```
class Solution:
    def threeSumMulti(self, arr, target):
        arr.sort()
        answer = 0
        modulo = 10**9 + 7
        for i in range(len(arr)):
            left, right = i + 1, len(arr) - 1
            while left < right:
                total = arr[i] + arr[left] + arr[right]
                if total < target:
                    left += 1
                elif total > target:
                    right -= 1
                elif arr[left] != arr[right]:
                    left_count = 1
                    right_count = 1
                    while left + 1 < right and arr[left] == arr[left + 1]:
                        left += 1
                        left_count += 1
                    while right - 1 > left and arr[right] == arr[right - 1]:
                        right -= 1
                        right_count += 1
                    answer += left_count * right_count
                    left += 1
                    right -= 1
                else:
                    count = right - left + 1
                    answer += count * (count - 1) // 2
                    break
        return answer % modulo
```

## Problem: 925. Long Pressed Name - Easy

```
class Solution:
    def isLongPressedName(self, name, typed):
        i = j = 0
        while j < len(typed):
            if i < len(name) and name[i] == typed[j]:
                i += 1
            elif j == 0 or typed[j] != typed[j - 1]:
                return False
            j += 1
```

```
j += 1
return i == len(name)
```

## Problem: 942. DI String Match - Easy

```
class Solution:
    def diStringMatch(self, s):
        low, high = 0, len(s)
        result = []
        for character in s:
            if character == 'I':
                result.append(low)
                low += 1
            else:
                result.append(high)
                high -= 1
        result.append(low)
        return result
```

## Problem: Matrix-Vector Dot Product

Write a Python function that computes the dot product of a matrix and a vector. The function should return a list representing the resulting vector if the operation is valid, or `-1` if the matrix and vector dimensions are incompatible. A matrix, represented as a list of lists, can be dotted with a vector only if the number of columns in the matrix equals the length of the vector. For example, an `n x m` matrix requires a vector of length `m`.

```
def matrix_dot_vector(a: list[list[int | float]], b: list[int | float]) ->
list[int | float]:
    if not a:
        return []
    vector_length = len(b)
    if any(len(row) != vector_length for row in a):
        return -1
    return [
        sum(value * weight for value, weight in zip(row, b))
        for row in a
    ]
```